

소스 변경 상세 — 무엇이 어떻게 바뀌었나

①번 문서의 파일 27개가 실제로 어떻게 바뀌는지 전·후로 보여 줍니다. 파일을 통째로 교체하실 거면 6절만 보셔도 되고, 손으로 고치거나 대조 검수를 하실 때 2~5절을 쓰십시오. 반영할 파일 10개의 전문은 맨 끝 「별첨1·2·3」에 있습니다(자바 6 · 매퍼 2 · 스키마 2).

작성일 2026-09-08 (확정본) 기준 내부망 반입 zip(2026-09-07) 대비 선행 문서 ① 스키마 변경

검증 mvn -o clean test 62건 통과

● 이 문서의 SQL 에는 스키마 접두어 INST1. 가 붙어 있습니다

사내 표준이 FROM INST1.TSKGIAF01 처럼 스키마를 명시하도록 요구합니다. 그런데 외부망 리포의 실제 파일에는 아직 접두어가 없습니다 — 표준 요구를 문서(=내부망용)에만 먼저 반영한 것입니다(사용자 확정 2026-09-08).

따라서 두 가지를 유념하십시오:

- 내부망 파일과 리포 파일이 갈립니다. 내부망에서는 이 문서(별첨)를 기준으로 삼으십시오 — 리포에서 그냥 복사해 오면 접두어가 빠집니다.
- ⚠ import_wtp.py 로 내부망 파일을 리포로 되돌리면 접두어가 리포로 들어와 외부망 테스트(H2)가 전부 깨집니다. 되돌릴 때는 접두어를 빼거나, 리포에도 함께 반영할지 먼저 정하십시오.
- MySQL 미러(schema-mysql.sql)에는 붙이지 않았습니다 — 외부망 테스트 전용이라 INST1 스키마가 없습니다.

1 왜 자바까지 바뀌나 — TypeHandler

MyBatis의 resultMap 은 자바 필드와 DB 컬럼을 잇는 번역표이고, TypeHandler는 그 번역표가 값을 어떻게 바꿀지 정하는 규칙입니다. 표준이 요구하는 값(문자열 시각·2자리 코드)과 자바가 쓰는 값이 다르므로, 변환을 세 클래스에 가두어 도메인·서비스·화면·테스트를 지켰습니다.

TypeHandler	자바가 쓰는 값	DB 에 들어가는 값
LocalDateTimeStringTypeHandler	LocalDateTime	'20260908103000'
ClassificationStatusTypeHandler	"UNCLASSIFIED"	'01'
InstitutionCategoryTypeHandler	"지방자치단체"	'01'

규칙 두 개만 지키면 됩니다

1. resultMap 과 #{...} 양쪽에 적는다. 두 코드 핸들러가 모두 자바 String 을 다뤄 자동 등록을 쓸 수 없습니다. 한쪽만 적으면 읽거나 쓰기 중 한 방향만 변환돼 조용히 어긋납니다.
2. SQL 에 직접 박은 값은 코드로 적는다. TypeHandler 는 파라미터와 결과에만 걸립니다. SET 분류 상태구분 = '04' 자리에 'DELETED' 를 적으면 오류 없이 0건이 바뀝니다.

2 신규 자바 9개 — 무엇을 하는 파일인가

파일	역할	고칠 일이 생기면
code/InstitutionCategory.java	기관영업분류 코드↔이름 5쌍. 화면 드롭다운의 유일한 출처	분류를 늘릴 때 이 파일만
code/ClassificationStatus.java	분류상태 코드↔값 4쌍 + 상수	상태를 늘릴 때(도메인도 함께)
code/SystemUser.java	화면번호 도출 + ThreadLocal. KGI11100\$UploadAction → GI11100	배치 코드를 늘릴 때
config/CodeTypeHandler.java	코드 변환 공통 뼈대(추상). 모르는 값이면 예외	거의 없음
config/InstitutionCategoryTypeHandler.java	위 뼈대 + 기관영업분류 표	없음
config/ClassificationStatusTypeHandler.java	위 뼈대 + 분류상태 표	없음
config/LocalDateTimeStringTypeHandler.java	yyyyMMddHHmmss 변환. 읽을 때 trim()	일시 형식이 바뀌면
config/SystemUserInterceptor.java	요청마다 화면번호를 넣고 끝나면 지움	없음
config/WebMvcConfig.java	인터셉터 등록	인터셉터가 늘 때

ThreadLocal 은 반드시 지워야 합니다

WAS 는 스레드를 재사용합니다. SystemUserInterceptor.afterCompletion 과 재분류 배치의 finally 에서 **지우지 않으면 다음 작업이 이전 화면번호를 물려받아 감사 기록이 조용히 틀어집니다.** 두 곳 모두 이미 지우고 있습니다 — 새 진입점을 만들 때 같은 규칙을 지키십시오.

모르는 값이 오면 조용히 넘기지 않습니다

ClassificationStatusTypeHandler: 코드로 옮길 수 없는 값이다 - '지자체'. 아는 값: [...]
 InstitutionCategoryTypeHandler: DB 에 모르는 코드가 들어 있다 - '99'. 아는 코드: [01, 02, ...]
 일시 형식이 YYYYMMDDHH24MISS 가 아니다 - '2026-09-08 10:30:00'

코드가 어긋나면 증상이 “저장은 됐는데 조회가 안 됨”처럼 엉뚱하게 나타나므로, **그 자리에서 예외로 멈추게** 했습니다. 위 메시지가 보이면 원인이 바로 드러납니다.

3 UploadedFileMapper.xml

3-1. resultMap 9행 전부 + TypeHandler 4곳

```
전 <id      property="id"           column="ID"/>
<result property="originalName"    column="ORIGINAL_NAME"/>
<result property="storedPath"      column="STORED_PATH"/>
<result property="year"            column="FILE_YEAR"/>
<result property="institutionName" column="INSTITUTION_NAME"/>
<result property="category"        column="CATEGORY"/>
<result property="status"          column="STATUS"/>
<result property="uploadedAt"      column="UPLOADED_AT"/>
<result property="classifiedAt"    column="CLASSIFIED_AT"/>
```

```
후 <id      property="id"           column="업로드파일일련번호"/>
<result property="institutionName" column="기관명"/>
<result property="category"        column="기관영업분류코드"
      typeHandler="com.kb.uploader.config.InstitutionCategoryTypeHandler"/>
<result property="year"            column="문서년"/>
<result property="status"          column="분류상태구분"
      typeHandler="com.kb.uploader.config.ClassificationStatusTypeHandler"/>
<result property="originalName"    column="원본파일명"/>
<result property="uploadedAt"      column="업로드일시"
      typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
<result property="classifiedAt"    column="분류일시"
      typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
<result property="storedPath"      column="저장경로"/>
```

3-2. <sql id="값목록"> 신설 — 파라미터 쪽 표기가 길어 묶었다

```
신설 <sql id="값목록">
    #{institutionName},
    #{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler},
    #{year},
    #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},
    #{originalName},
    #{uploadedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
    #{storedPath}
</sql>
```

3-3. insert oracle — 내부망이 실제로 쓰는 쪽

```
전 SELECT UPLOADED_FILE_SEQ.NEXTVAL FROM DUAL
INSERT INTO UPLOADED_FILE
    (ID, ORIGINAL_NAME, STORED_PATH, FILE_YEAR, INSTITUTION_NAME, CATEGORY, STATUS, UPLOADED_AT)
VALUES
    (#{id}, #{originalName}, #{storedPath}, #{year}, #{institutionName}, #{category}, #{status}, #{uploadedAt})
```

```
후 SELECT INST1.SQ_KGI_AGENTFILETABLE.NEXTVAL FROM DUAL
INSERT INTO INST1.TSKGIAF01
    (업로드파일일련번호, 기관명, 기관영업분류코드, 문서년, 분류상태구분, 원본파일명, 업로드일시, 저장경로)
VALUES
    (#{id}, <include refid="값목록"/>)
```

⚠ 시퀀스 이름도 바뀝니다. 여기가 틀리면 ORA-02289 로 업로드만 실패합니다.

3-4. update

```
전 UPDATE UPLOADED_FILE
SET STORED_PATH = #{storedPath}, INSTITUTION_NAME = #{institutionName},
    FILE_YEAR = #{year}, CATEGORY = #{category},
    STATUS = #{status}, CLASSIFIED_AT = #{classifiedAt}
WHERE ID = #{id}
```

```
후 UPDATE INST1.TSKGIAF01
SET 저장경로 = #{storedPath},
    기관명 = #{institutionName},
    문서년 = #{year},
    기관영업분류코드 = #{category,typeHandler=...InstitutionCategoryTypeHandler},
    분류상태구분 = #{status,typeHandler=...ClassificationStatusTypeHandler},
    분류일시 = #{classifiedAt,typeHandler=...LocalDateTimeStringTypeHandler}
WHERE 업로드파일일련번호 = #{id}
```

3-5. softDeleteById · rejectById 값을 코드로

```
전 UPDATE UPLOADED_FILE SET STATUS = 'DELETED' WHERE ID = #{id}
UPDATE UPLOADED_FILE SET STATUS = 'REJECTED' WHERE ID = #{id}
```

```
후 UPDATE INST1.TSKGIAF01 SET 분류상태구분 = '04' WHERE 업로드파일일련번호 = #{id}
UPDATE INST1.TSKGIAF01 SET 분류상태구분 = '03' WHERE 업로드파일일련번호 = #{id}
```

⚠ **여기가 이번 변경에서 가장 놓치기 쉬운 자리입니다.** 리터럴은 TypeHandler 를 타지 않아 'DELETED' 를 그대로 두면 오류 없이 아무 행동 안 바꿉니다.

3-6. findByStatus 정렬 신설 + 파라미터 변환

```
전 SELECT * FROM UPLOADED_FILE WHERE STATUS = #{status}
```

```
후 SELECT * FROM INST1.TSKGIAF01
WHERE 분류상태구분 = #{status,typeHandler=...ClassificationStatusTypeHandler}
ORDER BY 업로드파일일련번호 DESC
```

종전에는 ORDER BY 가 없었습니다. 목록 순서를 DB 에 맡긴 셈이라 삽입순처럼 보였을 뿐 보장이 아니었고, 인덱스가 붙는 순간 순서가 바뀝니다. 이 문장은 앱에서 가장 많이 쓰입니다 — 파일처리상태 화면·분류완료 목록 화면·재분류 잡·InstitutionService 두 곳, 모두 여섯 자리.

3-7. findRecent 정렬 기준을 PK 로

```
전 SELECT * FROM (
    SELECT * FROM UPLOADED_FILE ORDER BY UPLOADED_AT DESC
) WHERE ROWNUM <= #{limit}
```

```
후 SELECT * FROM (
    SELECT * FROM INST1.TSKGIAF01 ORDER BY 업로드파일일련번호 DESC
) WHERE ROWNUM <= #{limit}
```

업로드일시가 문자열 초 단위라 같은 초에 여러 건이면 순서가 뒤섞입니다. PK 는 그 경우에도 정확하고, PK 인덱스가 정렬을 그대로 처리합니다.

3-8. 나머지 countAll · countByStatus · findClassifiedByUnknownInstitution · 페이징·검색

```
전 SELECT COUNT(*) FROM UPLOADED_FILE
SELECT COUNT(*) FROM UPLOADED_FILE WHERE STATUS = #{status}
... WHERE INSTITUTION_NAME = '알수없음' AND STATUS = 'CLASSIFIED' ORDER BY UPLOADED_AT DESC
... WHERE INSTITUTION_NAME LIKE '%' || #{keyword} || '%' ORDER BY UPLOADED_AT DESC
... AND (#{year} IS NULL OR FILE_YEAR = #{year})
... AND (#{keyword} IS NULL OR ORIGINAL_NAME LIKE '%' || #{keyword} || '%')
```

```
후 SELECT COUNT(*) FROM INST1.TSKGIAF01
SELECT COUNT(*) FROM INST1.TSKGIAF01 WHERE 분류상태구분 = #{status,typeHandler=...}
... WHERE 기관명 = '알수없음' AND 분류상태구분 = '02' ORDER BY 업로드일시 DESC
... WHERE 기관명 LIKE '%' || #{keyword} || '%' ORDER BY 업로드일시 DESC
... AND (#{year} IS NULL OR 문서년 = #{year})
... AND (#{keyword} IS NULL OR 원본파일명 LIKE '%' || #{keyword} || '%')
```

검색·페이징은 일부러 업로드일시 정렬을 유지했습니다

인덱스 (기관명, 업로드일시) 의 두 번째 열이 업로드일시라, 기관 입력이 드롭다운(정확 일치)이 되는 날 정렬까지 인덱스가 받게 하려는 것입니다. 여기를 일련번호로 바꾸면 그 이점이 사라집니다.

4 InstitutionMapper.xml

4-1. resultMap

```
전 <id      property="id"          column="ID"/>
<result property="name"          column="NAME"/>
<result property="category"      column="CATEGORY"/>
<result property="modifiedAt"    column="MODIFIED_AT"/>
```

```
후 <id      property="id"          column="기관일련번호"/>
<result property="name"          column="기관명"/>
<result property="category"      column="기관영업분류코드"
      typeHandler="com.kb.uploader.config.InstitutionCategoryTypeHandler"/>
<result property="modifiedAt"    column="수정일시"
      typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
```

4-2. 나머지 7문장

```
전 SELECT INSTITUTION_SEQ.NEXTVAL FROM DUAL
INSERT INTO INSTITUTION (ID, NAME, CATEGORY, MODIFIED_AT) VALUES ({id}, {name}, {category}, {modifiedAt})
INSERT INTO INSTITUTION (NAME, CATEGORY, MODIFIED_AT)                                -- mysql
UPDATE INSTITUTION SET NAME = {name}, CATEGORY = {category}, MODIFIED_AT = {modifiedAt} WHERE ID = {id}
DELETE FROM INSTITUTION WHERE ID = {id}
DELETE FROM INSTITUTION
SELECT * FROM INSTITUTION WHERE NAME = {name}
SELECT * FROM INSTITUTION ORDER BY NAME ASC
```

```
후 SELECT INST1.SQ_KGI_AGENTGIGANTABLE.NEXTVAL FROM DUAL
INSERT INTO INST1.TSKGIAF02 (기관일련번호, 기관명, 기관영업분류코드, 수정일시) VALUES ({id}, <include refid="값목록"/>)
INSERT INTO INST1.TSKGIAF02 (기관명, 기관영업분류코드, 수정일시)                                -- mysql
1
UPDATE INST1.TSKGIAF02 SET 기관명 = {name}, 기관영업분류코드 = {category,typeHandler=...},
                        수정일시 = {modifiedAt,typeHandler=...} WHERE 기관일련번호 = {id}
DELETE FROM INST1.TSKGIAF02 WHERE 기관일련번호 = {id}
DELETE FROM INST1.TSKGIAF02
SELECT * FROM INST1.TSKGIAF02 WHERE 기관명 = {name}
SELECT * FROM INST1.TSKGIAF02 ORDER BY 기관명 ASC
```

`findByName` 이 이 앱에서 가장 중요한 문장입니다

분류가 여기서 결정됩니다 — 업로드된 파일의 기관명으로 이 문장을 돌려 못 찾으면 파일이 미분류로 남습니다. 틀리면 오류 없이 “분류가 하나도 안 되는” 증상으로만 나타납니다.

TSKGIAF02 에 컬럼 4개가 늘었지만 매퍼는 바뀌지 않습니다

협약기관상태구분 · 입찰공고상태 · 입찰공고시작 · 입찰공고마감 이 신설됐으나 애플리케이션이 읽지도 쓰지도 않습니다. resultMap 은 모르는 컬럼을 그냥 무시하고 INSERT 는 컬럼을 명시하므로 NULL 로 남습니다 — 이 절과 4절에 추가로 손댈 것이 없습니다.

5 컨트롤러 3개 + 화면 1개

카테고리 목록이 세 곳에 복제돼 있었고 값이 옛 4종이었습니다. 새 5종과 다르므로 그대로 두면 `ExceptionHandler` 가 기관 등록을 거부합니다.

`KGI12000$FileStatusList.java` · `KGI12400$ClassifiedFileList.java`

```
전 private static final List<String> ALL_CATEGORIES =  
    Arrays.asList("지자체", "대학교", "대학병원", "공공기관");  
...  
model.addAttribute("categories", ALL_CATEGORIES);
```

```
후 import com.kb.uploader.code.InstitutionCategory;  
...  
model.addAttribute("categories", InstitutionCategory.names());
```

상수 선언과 `import java.util.Arrays;` 는 지웁니다.

`KGI13000$InstitutionList.java` 화면에 목록을 넘기도록 추가

```
추가 import com.kb.uploader.code.InstitutionCategory;  
...  
model.addAttribute("categories", InstitutionCategory.names());
```

`templates/institution.html`

```
전 <option value="지자체">지자체</option>  
<option value="대학교">대학교</option>  
<option value="대학병원">대학병원</option>  
<option value="공공기관">공공기관</option>
```

```
후 <option th:each="cat : ${categories}" th:value="${cat}" th:text="${cat}"></option>
```

다른 화면(`upload.html` · `file-status.html` · `classified-rework.html`)은 이미 `${categories}` 를 쓰고 있어 손댈 것이 없습니다.

6 내부망에서 적용하는 법

방법 ㉓ — 파일 교체 권장

외부망 리포	내부망 WTP 폴더
uploader/src/main/java/com/kb/uploader/**/*.java	src/com/kb/uploader/**/*.java
uploader/src/main/resources/mapper/*.xml	src/mapper/*.xml
uploader/src/main/resources/schema-*.sql	src/schema-*.sql
uploader/src/main/resources/templates/institution.html	src/templates/institution.html

신규 폴더 `src/com/kb/uploader/code/` 가 생깁니다. 교체 후 **Project** → **Clean** → **Publish**.

방법 ㉔ — 매퍼를 손으로 고칠 때의 치환 순서

순서를 지키지 않으면 XML 이 깨집니다

INSTITUTION → TSKGIAF02 를 먼저 돌리면 INSTITUTION_NAME · INSTITUTION_SEQ 까지 망가집니다.
ID 는 databaseId · keyProperty 같은 속성 이름에도 들어 있어 무차별 치환하면 문서가 통째로 깨 집니다.

#	찾을 것	바꿀 것	주의
1	UPLOADED_FILE_SEQ	SQ_KGI_AGENTFILETABLE	가장 길어서 먼저
2	INSTITUTION_SEQ	SQ_KGI_AGENTGIGANTABLE	—
3	INSTITUTION_NAME	기관명	INSTITUTION 보다 먼저
4	ORIGINAL_NAME	원본파일명	—
5	CLASSIFIED_AT	분류일시	—
6	MODIFIED_AT	수정일시	—
7	UPLOADED_AT	업로드일시	—
8	STORED_PATH	저장경로	—
9	FILE_YEAR	문서년	—
10	UPLOADED_FILE	TSKGIAF01	1번을 먼저 했어야 안전
11	INSTITUTION	TSKGIAF02	2·3번을 먼저 했어야 안전
12	CATEGORY	기관영업분류코드	한 번만
13	STATUS	분류상태구분	한 번만
14	NAME	기관명	⚠ InstitutionMapper 에서만
15	ID	업로드파일일련번호 / 기관일련번호	🔴 일괄 치환 금지 — 한 곳씩 눈으로
16	그다음 TypeHandler 를 손으로 넣는다(3-1·3-2·4-1) — 치환으로는 안 됩니다		
17	그다음 리터럴을 코드로(3-5의 '04' · '03' , 3-8의 '02')		

16·17번 때문에 ㉔는 권하지 않습니다. 파일 교체가 훨씬 안전합니다.

7 적용 후 확인

확인	방법	정상
컴파일	Project → Clean 후 Problems 뷰	오류 0건. <code>code</code> 패키지가 보임
배포	<code>WEB-INF/classes/mapper/*.xml</code> 열어보기	<code>TSKGIAF01</code> 이 보임
기동	서버 로그	TypeHandler 예외 없음
드롭다운	<code>/uploader/institutions</code>	분류 선택지가 5종 (지방자치단체·공공기관·대학교·병원·법원)
코드 저장	테이블 생성 후 기관 1건 등록 → DB 조회	기관영업분류코드 에 <code>'01'</code> 같은 코드
일시 저장	파일 1건 업로드 → DB 조회	업로드일시 가 <code>20260908103000</code> 14자

별첨1 신규 자바 9개 — 소스 전문

파일을 그대로 복사하실 수 있으면 이 별첨은 보지 않으셔도 됩니다. 반입 경로가 막혀 손으로 옮겨 적어야 하거나, 반입 전 검수가 필요할 때 쓰십시오.

옮겨 적을 때 지킬 것 셋

- 파일 인코딩은 UTF-8 — 주석·문자열에 한글이 많아 cp949 로 저장하면 통째로 깨집니다.
- 패키지 경로 = 폴더 경로. `package com.kb.uploader.code;` → `src/com/kb/uploader/code/` . ⚠
`code` 폴더는 새로 만들어야 합니다.
- 클래스 이름 = 파일 이름(대소문자 포함).

`src/com/kb/uploader/code/InstitutionCategory.java` 57줄

```
package com.kb.uploader.code;

import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * 기관영업분류코드 - 사내 인포타임 {@code 코드2} 로 저장하는 코드값.
 *
 * <p><b>왜 코드인가.</b> 사내 DB 표준의 {@code 구분코드} 는 <b>5자리 이하</b>여야 한다
 * (DBA 정책). 그런데 우리가 쓰던 값 {@code 지방자치단체} 는 6자리 들어가지 않는다.
 * 그래서 DB 에는 {@code '01'} 을 넣고, <b>자바와 화면은 한글 이름을 그대로 쓴다.</b>
 * 변환은 {@code InstitutionCategoryTypeHandler} 한 곳에서만 일어난다 -
 * 도메인·서비스·컨트롤러·Thymeleaf 는 이 코드의 존재를 모른다.
 *
 * <p>⚠ <b>여기가 카테고리 목록의 유일한 출처다.</b> 종전에는 같은 목록이
 * {@code KGI12000}·{@code KGI12400} 두 컨트롤러와 {@code institution.html} 에
 * <b>세 벌</b>로 복제돼 있었고, 그래서 옛 4종(지자체·대학교·대학병원·공공기관)이
 * 그대로 남아 있었다. 늘리거나 바꿀 때는 이 파일만 고친다.
 *
 * <p>⚠ 코드값을 바꾸면 <b>이미 저장된 행을 함께 옮겨야 한다.</b> 지금은 테이블이
 * 비어 있어 자유롭지만, 운영 뒤에는 UPDATE 가 따른다.
 */
public final class InstitutionCategory {

    private static final Map<String, String> NAME_TO_CODE = new LinkedHashMap<String, String>();
    private static final Map<String, String> CODE_TO_NAME = new LinkedHashMap<String, String>();

    static {
        put("01", "지방자치단체");
        put("02", "공공기관");
        put("03", "대학교");
        put("04", "병원");
        put("05", "법원");
    }

    private static void put(String code, String name) {
        NAME_TO_CODE.put(name, code);
        CODE_TO_NAME.put(code, name);
    }

    /** 화면 드롭다운에 그대로 쓰는 순서다(코드 오름차순). */
    public static List<String> names() {
        return Collections.unmodifiableList(new java.util.ArrayList<String>(NAME_TO_CODE.keySet()));
    }

    public static Map<String, String> nameToCode() {
        return Collections.unmodifiableMap(NAME_TO_CODE);
    }
}
```

```

    }

    public static Map<String, String> codeToName() {
        return Collections.unmodifiableMap(CODE_TO_NAME);
    }

    private InstitutionCategory() {}
}

```

src/com/kb/uploader/code/ClassificationStatus.java 51줄

```

package com.kb.uploader.code;

import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * 분류상태 - 사내 인포타입 {@code 구분코드2} 로 저장하는 코드값.
 *
 * <p>자바가 쓰는 값은 {@code "UNCLASSIFIED"} 처럼 <b>종전 그대로</b>다
 * ({@code UploadedFile.classify()}·{@code softDelete()} 등). 12자리 5자리 상한을
 * 넘으므로 DB 에만 코드로 바꿔 넣는다 - 변환은
 * {@code ClassificationStatusTypeHandler} 한 곳뿐이다.
 *
 * <p>⚠ <b>Mapper XML 의 SQL 리터럴은 이 변환을 타지 않는다.</b>
 * TypeHandler 는 <b>파라미터와 결과</b>에만 걸리므로,
 * {@code SET 분류상태구분 = '04'} 처럼 문장에 직접 박힌 값은 <b>코드로 적어야 한다.</b>
 * 그 자리에 {@code 'DELETED'} 를 적으면 조용히 아무 행동도 안 바뀐다.
 */
public final class ClassificationStatus {

    public static final String UNCLASSIFIED = "UNCLASSIFIED";
    public static final String CLASSIFIED = "CLASSIFIED";
    public static final String REJECTED = "REJECTED";
    public static final String DELETED = "DELETED";

    private static final Map<String, String> NAME_TO_CODE = new LinkedHashMap<String, String>();
    private static final Map<String, String> CODE_TO_NAME = new LinkedHashMap<String, String>();

    static {
        put("01", UNCLASSIFIED);
        put("02", CLASSIFIED);
        put("03", REJECTED);
        put("04", DELETED);
    }

    private static void put(String code, String name) {
        NAME_TO_CODE.put(name, code);
        CODE_TO_NAME.put(code, name);
    }

    public static Map<String, String> nameToCode() {
        return Collections.unmodifiableMap(NAME_TO_CODE);
    }

    public static Map<String, String> codeToName() {
        return Collections.unmodifiableMap(CODE_TO_NAME);
    }

    private ClassificationStatus() {}
}

```

src/com/kb/uploader/code/SystemUser.java 61줄

```

package com.kb.uploader.code;

/**

```

```

* 감사 컬럼 {@code 시스템사용자번호} 에 넣을 값 - <b>이 데이터를 넣게 만든 주체</b>다.
*
* <p>이 앱에는 로그인도 사용자 개념도 없다. 그래서 주체는 사람이 아니라 <b>프로그램</b>이고,
* 화면번호를 넣는다. 인포타입이 {@code 사용자번호7}(CHAR 7) 이라 <b>앞의 {@code K} 를 뺀
* 7자</b>를 쓴다 - {@code KGI11100} → {@code GI11100}.
*
* <p><b>값을 손으로 심지 않는다.</b> 컨트롤러 클래스명이 이미 화면번호를 담고 있으므로
* ({@code KGI11100$UploadAction}) 거기서 뺏는다 - {@code SystemUserInterceptor} 가
* 요청마다 넣고 끝나면 지운다. <b>화면이 늘어도 여기 손댈 일이 없다.</b>
*
* <p>⚠ 웹 요청이 아닌 경로(재분류 배치)는 컨트롤러가 없으므로 {@link #BATCH} 를
* <b>직접 넣고 반드시 지워야</b> 한다. ThreadLocal 이라 안 지우면 그 스레드를 물려받은
* 다음 작업에 값이 새어 나간다.
*
* <p>⚠ 값이 없으면 {@code null} 이다. 컬럼이 NULLABLE 이라 기동은 되지만, 그 행은
* "누가 넣었는지 모르는 행"이 된다 - 새 쓰기 경로를 만들 때 컨텍스트가 세팅되는지 확인할 것.
*/
public final class SystemUser {

    /** 재분류 배치처럼 화면이 없는 경로. 7자를 넘지 않는다. */
    public static final String BATCH = "BATCH01";

    private static final ThreadLocal<String> CURRENT = new ThreadLocal<String>();

    /** 컨트롤러 클래스명에서 화면번호를 뺏는다. {@code KGI11100$UploadAction} → {@code GI11100}. */
    public static String ofController(Class<?> controller) {
        if (controller == null) {
            return null;
        }
        String name = controller.getSimpleName();
        // 규약: K + GI + 5자리. 뒤에 $이름 이 붙는다.
        if (name.length() < 8 || name.charAt(0) != 'K') {
            return null; // FileSearchApiController 처럼 번호가 없는 것 - 읽기 전용이라 무방하다
        }
        String code = name.substring(1, 8);
        for (int i = 2; i < code.length(); i++) {
            if (!Character.isDigit(code.charAt(i))) {
                return null;
            }
        }
        return code;
    }

    public static void set(String systemUserNo) {
        CURRENT.set(systemUserNo);
    }

    /** 세팅된 값이 없으면 {@code null} - 호출부가 판단하지 않고 그대로 DB 에 넣는다. */
    public static String get() {
        return CURRENT.get();
    }

    public static void clear() {
        CURRENT.remove();
    }

    private SystemUser() {}
}

```

src/com/kb/uploader/config/CodeTypeHandler.java 74줄

```

package com.kb.uploader.config;

import org.apache.ibatis.type.BaseTypeHandler;
import org.apache.ibatis.type.JdbcType;

import java.sql.CallableStatement;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

```

```

import java.util.Map;

/**
 * 자바가 쓰는 <b>이름</b>과 DB 에 들어가는 <b>코드</b>를 잇는다.
 *
 * <p><b>왜 두는가.</b> 사내 DB 표준의 {@code 구분코드} 는 5자리 이하여야 하는데
 * 우리가 쓰는 값({@code UNCLASSIFIED}·{@code 지방자치단체})은 그보다 길다.
 * 값을 코드로 바꾸면 도메인·서비스·컨트롤러·화면·테스트가 전부 바뀌므로,
 * <b>변환을 여기 한 곳에 가둔다.</b> 그 덕에 자바 쪽은 종전 값을 그대로 쓴다.
 *
 * <p>⚠ <b>모르는 값이 오면 조용히 넘기지 않고 예외를 던진다.</b> 코드가 안 맞으면
 * 증상이 "저장은 됐는데 조회가 안 됨" 처럼 엉뚱하게 나타나기 때문이다.
 *
 * <p>⚠ 두 하위 구현 모두 자바 타입이 {@code String} 이라 <b>자동 등록(type-handlers-package)
 * 을 쓸 수 없다</b> - 같은 타입에 둘이 걸려 어느 쪽이 잡힐지 알 수 없다.
 * 그래서 Mapper XML 에서 {@code typeHandler=} 로 <b>자리마다 명시</b>한다.
 */
public abstract class CodeTypeHandler extends BaseTypeHandler<String> {

    protected abstract Map<String, String> nameToCode();

    protected abstract Map<String, String> codeToName();

    @Override
    public void setNonNullParameter(PreparedStatement ps, int i, String name, JdbcType jdbcType)
        throws SQLException {
        String code = nameToCode().get(name);
        if (code == null) {
            throw new SQLException(getClass().getSimpleName()
                + ": 코드로 옮길 수 없는 값이다 - '" + name + "'. 아는 값: " + nameToCode().keySet());
        }
        ps.setString(i, code);
    }

    @Override
    public String getNullableResult(ResultSet rs, String columnName) throws SQLException {
        return toName(rs.getString(columnName));
    }

    @Override
    public String getNullableResult(ResultSet rs, int columnIndex) throws SQLException {
        return toName(rs.getString(columnIndex));
    }

    @Override
    public String getNullableResult(CallableStatement cs, int columnIndex) throws SQLException {
        return toName(cs.getString(columnIndex));
    }

    private String toName(String code) throws SQLException {
        if (code == null) {
            return null;
        }
        String trimmed = code.trim();
        if (trimmed.isEmpty()) {
            return null;
        }
        String name = codeToName().get(trimmed);
        if (name == null) {
            throw new SQLException(getClass().getSimpleName()
                + ": DB 에 모르는 코드가 들어 있다 - '" + trimmed + "'. 아는 코드: " + codeToName().keySet());
        }
        return name;
    }
}

```

src/com/kb/uploader/config/InstitutionCategoryTypeHandler.java 19줄

```
package com.kb.uploader.config;
```

```

import com.kb.uploader.code.InstitutionCategory;

import java.util.Map;

/** {@code 지방자치단체} ↔ {@code '01'} - 자세한 이유는 {@link CodeTypeHandler}. */
public class InstitutionCategoryTypeHandler extends CodeTypeHandler {

    @Override
    protected Map<String, String> nameToCode() {
        return InstitutionCategory.nameToCode();
    }

    @Override
    protected Map<String, String> codeToName() {
        return InstitutionCategory.codeToName();
    }
}

```

src/com/kb/uploader/config/ClassificationStatusTypeHandler.java 19줄

```

package com.kb.uploader.config;

import com.kb.uploader.code.ClassificationStatus;

import java.util.Map;

/** {@code UNCLASSIFIED} ↔ {@code '01'} - 자세한 이유는 {@link CodeTypeHandler}. */
public class ClassificationStatusTypeHandler extends CodeTypeHandler {

    @Override
    protected Map<String, String> nameToCode() {
        return ClassificationStatus.nameToCode();
    }

    @Override
    protected Map<String, String> codeToName() {
        return ClassificationStatus.codeToName();
    }
}

```

src/com/kb/uploader/config/LocalDateTimeStringTypeHandler.java 70줄

```

package com.kb.uploader.config;

import org.apache.ibatis.type.BaseTypeHandler;
import org.apache.ibatis.type.JdbcType;

import java.sql.CallableStatement;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;

/**
 * 자바 {@link LocalDateTime} 과 사내 인포타입 {@code 일시20}(CHARACTER 20) 을 잇는다.
 *
 * <p>저장 형식은 <b>{@code YYYYMMDDHH24MISS}</b> 14자다(사내 표준). 컬럼이 20자인 것은
 * 여윌 뿐이라 우리는 14자만 채운다.
 *
 * <p><b>왜 두는가.</b> 표준이 시각을 문자열로 두라고 하는데 도메인은
 * {@code LocalDateTime} 이다. 도메인을 문자열로 바꾸면 {@code UploadedFile}·서비스·
 * 화면·테스트가 전부 바뀌므로, <b>변환을 여기 한 곳에 가둔다.</b>
 *
 * <p>⚠ 읽을 때 {@code trim()} 한다. 컬럼이 고정폭 {@code CHAR} 로 만들어지면
 * 뒤에 공백이 붙어 오는데, 그대로 파싱하면 {@code DateTimeParseException} 이 난다.
 *

```

```

* <p>⚠ <b>정렬이 문자열 정렬</b>이 된다. {@code YYYYMMDDHH24MISS} 는 자릿수가 고정이라
* 사전순 = 시간순이 맞지만, 형식을 바꾸면 그 성질이 깨진다
* ({@code ORDER BY 업로드일시 DESC} 가 조용히 어긋난다).
*/
public class LocalDateTimeStringTypeHandler extends BaseTypeHandler<LocalDateTime> {

    private static final DateTimeFormatter FORMAT = DateTimeFormatter.ofPattern("yyyyMMddHHmmss");

    @Override
    public void setNonNullParameter(PreparedStatement ps, int i, LocalDateTime parameter, JdbcType jdbcType)
        throws SQLException {
        ps.setString(i, parameter.format(FORMAT));
    }

    @Override
    public LocalDateTime getNullableResult(ResultSet rs, String columnName) throws SQLException {
        return parse(rs.getString(columnName));
    }

    @Override
    public LocalDateTime getNullableResult(ResultSet rs, int columnIndex) throws SQLException {
        return parse(rs.getString(columnIndex));
    }

    @Override
    public LocalDateTime getNullableResult(CallableStatement cs, int columnIndex) throws SQLException {
        return parse(cs.getString(columnIndex));
    }

    private LocalDateTime parse(String raw) throws SQLException {
        if (raw == null) {
            return null;
        }
        String value = raw.trim();
        if (value.isEmpty()) {
            return null;
        }
        try {
            return LocalDateTime.parse(value, FORMAT);
        } catch (DateTimeParseException e) {
            throw new SQLException("일시 형식이 YYYYMMDDHH24MISS 가 아니다 - '" + value + "'", e);
        }
    }
}

```

src/com/kb/uploader/config/SystemUserInterceptor.java 36줄

```

package com.kb.uploader.config;

import com.kb.uploader.code.SystemUser;
import org.springframework.stereotype.Component;
import org.springframework.web.method.HandlerMethod;
import org.springframework.web.servlet.HandlerInterceptor;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/**
 * 요청마다 {@link SystemUser} 에 화면번호를 넣고, 끝나면 지운다.
 *
 * <p>값은 <b>컨트롤러 클래스명에서 자동으로</b> 나온다({@code KGI11100$UploadAction} →
 * {@code KGI11100}). 컨트롤러마다 손으로 심지 않으므로 <b>화면이 늘어도 이 파일은 그대로다.</b>
 *
 * <p>⚠ {@code afterCompletion} 에서 <b>반드시 지운다.</b> WAS 는 스레드를 재사용하므로,
 * 안 지우면 다음 요청이 이전 화면번호를 물려받아 <b>감사 기록이 조용히 틀어진다.</b>
 */
@Component
public class SystemUserInterceptor implements HandlerInterceptor {

    @Override

```

```

    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) {
        if (handler instanceof HandlerMethod) {
            SystemUser.set(SystemUser.ofController(((HandlerMethod) handler).getBeanType()));
        }
        return true;
    }

    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse response,
                               Object handler, Exception ex) {
        SystemUser.clear();
    }
}

```

src/com/kb/uploader/config/WebMvcConfig.java 21줄

```

package com.kb.uploader.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

/** {@link SystemUserInterceptor} 를 모든 요청에 건다. */
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {

    private final SystemUserInterceptor systemUserInterceptor;

    public WebMvcConfig(SystemUserInterceptor systemUserInterceptor) {
        this.systemUserInterceptor = systemUserInterceptor;
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(systemUserInterceptor);
    }
}

```

별첨2매퍼 XML 2개 — 전문

이 둘은 손으로 고치는 것보다 통째로 옮기는 편이 훨씬 안전합니다

이름 치환만으로 끝나지 않습니다 — **TypeHandler** 를 자리마다 넣어야 하고, **SQL** 리터럴을 코드로 바꿔야 하며, 이번엔 **감사 2열**까지 붙였습니다. 빠뜨리면 오류 없이 조용히 어긋납니다.

XML 은 두 가지를 조심하십시오

- `<` 는 그대로 적어야 합니다 — 풀어 쓰면 태그로 읽혀 파일이 깨집니다.
- 첫 두 줄(`<?xml ...?>` · `<!DOCTYPE mapper ...>`)을 빠뜨리지 마십시오.

src/mapper/UploadedFileMapper.xml 232줄

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<!--
    ▲ **2026-09-08 - 사내 DB 표준을 반영했다.** 테이블은 코드, 컬럼은 한글, 그리고
    타입은 인포타입이 정한다. 자바 필드명(property)은 그대로다.

    UPLOADED_FILE → TSKGIAF01          시퀀스 → SQ_KGI_AGENTFILETABLE

    property      → 컬럼명              인포타입
    id             → 업로드파일일련번호  일련번호16 (DECIMAL)
    institutionName → 기관명              명40
    category       → 기관영업분류구분     코드2      ★ TypeHandler
    year          → 문서년                년4
    status         → 분류상태구분         구분코드2   ★ TypeHandler
    originalName   → 원본파일명           명260
    uploadedAt     → 업로드일시           일시14     ★ TypeHandler
    classifiedAt   → 분류일시             일시14     ★ TypeHandler
    storedPath     → 저장경로내용         내용600
    systemUsedAt   → 시스템사용일시       일시14     ★ TypeHandler
    systemUserNo   → 시스템사용자번호     사용자번호7

    ★ **TypeHandler 를 자리마다 명시한다.** 두 코드 핸들러가 모두 자바 `String` 을
    다뤄서 자동 등록(type-handlers-package)을 쓸 수 없다 - 같은 타입에 둘이 걸리면
    어느 쪽이 잡힐지 알 수 없다. 그래서 resultMap 과 `#{...}` 양쪽에 적는다.
    **한쪽만 적으면 읽거나 쓰기 중 한 방향만 변환돼 조용히 어긋난다.**

    ▲ **SQL 문장에 직접 박힌 값은 TypeHandler 를 타지 않는다.** 아래
    `분류상태구분 = '04'` 처럼 **코드로 적어야 한다**. 거기에 'DELETED' 를 적으면
    오류 없이 0건이 바뀐다. 코드표는 `code/ClassificationStatus.java` 에 있다.
-->
<mapper namespace="com.kb.uploader.mapper.UploadedFileMapper">

    <resultMap id="uploadedFileMap" type="UploadedFile">
        <id property="id" column="업로드파일일련번호"/>
        <result property="institutionName" column="기관명"/>
        <result property="category" column="기관영업분류구분"
            typeHandler="com.kb.uploader.config.InstitutionCategoryTypeHandler"/>
        <result property="year" column="문서년"/>
        <result property="status" column="분류상태구분"
            typeHandler="com.kb.uploader.config.ClassificationStatusTypeHandler"/>
        <result property="originalName" column="원본파일명"/>
        <result property="uploadedAt" column="업로드일시"
            typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
        <result property="classifiedAt" column="분류일시"
            typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
        <result property="storedPath" column="저장경로내용"/>
    </resultMap>
```

```

<!-- 파라미터 쪽 TypeHandler 표기가 길어 sql 조각으로 묶는다 -->
<sql id="값목록">
    #{institutionName},
    #{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler},
    #{year},
    #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},
    #{originalName},
    #{uploadedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
    #{storedPath}
</sql>

<!--
    ⚠ 같은 id 가 두 번 나오는 자리는 **방언 분기**다(databaseId). 접속한 DB 에 맞는
    쪽만 로드된다 - config/MyBatisConfig 가 값을 정한다.
    한쪽만 고치면 다른 DB 에서 조용히 낡는다. **항상 둘을 같이 고칠 것.**
-->

<!-- MySQL: useGeneratedKeys로 AUTO_INCREMENT id 채움 -->
<insert id="insert" databaseId="mysql" useGeneratedKeys="true" keyProperty="id">
    INSERT INTO INST1.TSKGIAF01
        (기관명, 기관영업분류구분, 문서년, 분류상태구분, 원본파일명, 업로드일시, 저장경로내용,
        시스템사용일시, 시스템사용자번호)
    VALUES
        (<include refid="값목록"/>,
        #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, #{systemUserNo})
</insert>

<!-- Oracle: 시퀀스에서 먼저 ID를 받아 직접 넣는다 (schema-oracle.sql 의 SQ_KGI_AGENTFILETABLE) -->
<insert id="insert" databaseId="oracle">
    <selectKey keyProperty="id" resultType="long" order="BEFORE">
        SELECT INST1.SQ_KGI_AGENTFILETABLE.NEXTVAL FROM DUAL
    </selectKey>
    INSERT INTO INST1.TSKGIAF01
        (업로드파일일련번호, 기관명, 기관영업분류구분, 문서년, 분류상태구분, 원본파일명, 업로드일시,
        저장경로내용, 시스템사용일시, 시스템사용자번호)
    VALUES
        ({id}, <include refid="값목록"/>,
        #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, #{systemUserNo})
</insert>

<update id="update">
    UPDATE INST1.TSKGIAF01
    SET
        저장경로내용      = #{storedPath},
        기관명           = #{institutionName},
        문서년           = #{year},
        기관영업분류구분 = #{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler},
        분류상태구분     = #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},
        분류일시         = #{classifiedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
        시스템사용일시   = #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
        시스템사용자번호 = #{systemUserNo}
    WHERE 업로드파일일련번호 = #{id}
</update>

<!--
    ⚠ '04' = DELETED, '03' = REJECTED, '02' = CLASSIFIED (code/ClassificationStatus.java)
    ⚠ 이 둘은 파라미터에 시각 값이 없어 **컨트롤러가 LocalDateTime.now() 를 넘긴다.**
    DB 함수(TO_CHAR(SYSDATE)/DATE_FORMAT(NOW()))를 쓰면 방언이 갈리고 테스트로
    확인하기도 어려워, 자바에서 정해 넘기는 쪽을 택했다.
-->

<update id="softDeleteById">
    UPDATE INST1.TSKGIAF01 SET 분류상태구분 = '04',
        시스템사용일시 = #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, 시스템사용
        자번호 = #{systemUserNo}
    WHERE 업로드파일일련번호 = #{id}
</update>

<update id="rejectById">
    UPDATE INST1.TSKGIAF01 SET 분류상태구분 = '03',
        시스템사용일시 = #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, 시스템사용
        자번호 = #{systemUserNo}

```

```

WHERE 업로드파일일련번호 = #{id}
</update>

<select id="findById" resultMap="uploadedFileMap">
    SELECT * FROM INST1.TSGIAF01 WHERE 업로드파일일련번호 = #{id}
</select>

<!--
    ⚠ **중전에는 ORDER BY 가 없었다**(2026-09-08 추가). 목록 순서를 DB 에 맡긴 셈이라
    화면이 대체로 삽입순처럼 보였을 뿐 보장된 것이 아니었고, 인덱스가 붙는 순간
    순서가 바뀐다. 정렬 컬럼을 PK 로 둔 이유는 findRecent 와 같다(같은 초 문제).
    인덱스 (분류상태구분, 업로드파일일련번호) 가 **필터와 정렬을 함께** 받는다.
-->
<select id="findByStatus" resultMap="uploadedFileMap">
    SELECT * FROM INST1.TSGIAF01
    WHERE 분류상태구분 = #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler}
    ORDER BY 업로드파일일련번호 DESC
</select>

<select id="findClassifiedByUnknownInstitution" resultMap="uploadedFileMap">
    SELECT * FROM INST1.TSGIAF01
    WHERE 기관명 = '알수없음' AND 분류상태구분 = '02'
    ORDER BY 업로드일시 DESC
</select>

<select id="countAll" resultType="long">
    SELECT COUNT(*) FROM INST1.TSGIAF01
</select>

<select id="countByStatus" resultType="long">
    SELECT COUNT(*) FROM INST1.TSGIAF01
    WHERE 분류상태구분 = #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler}
</select>

<!--
    ⚠ **최신순 정렬을 업로드일시가 아니라 PK 로 한다**(2026-09-08).
    ㉠ 시퀀스가 NOCACHE 라 증가순 = 삽입순 = 시간순이고,
    ㉡ 업로드일시는 문자열 초 단위라 **같은 초에 여러 건이면 순서가 뒤섞인다**,
    ㉢ **PK 인덱스가 정렬을 그대로 처리**해 (업로드일시) 인덱스가 필요 없어진다.
    ⚠ 시퀀스에 캐시를 켜면 ㉠ 가 깨진다 - 그때는 이 정렬을 다시 봐야 한다.
-->
<!-- MySQL: LIMIT -->
<select id="findRecent" databaseId="mysql" resultMap="uploadedFileMap">
    SELECT * FROM INST1.TSGIAF01 ORDER BY 업로드파일일련번호 DESC LIMIT #{limit}
</select>
<!-- Oracle: ROWNUM 은 ORDER BY 보다 먼저 매겨지므로 정렬을 인라인 뷰로 감싼다 -->
<select id="findRecent" databaseId="oracle" resultMap="uploadedFileMap">
    SELECT * FROM (
        SELECT * FROM INST1.TSGIAF01 ORDER BY 업로드파일일련번호 DESC
    ) WHERE ROWNUM <= #{limit}
</select>

<!--
    ⚠ 아래 세 문장(기관명 검색·검색)은 **업로드일시로 정렬한다** - findRecent 와 다르다.
    인덱스 (기관명, 업로드일시) 의 두 번째 열이 업로드일시라, 기관명이 정확 일치가
    되는 날 **정렬까지 인덱스가 처리**하게 하려는 것이다. 여기를 일련번호로 바꾸면
    그 이점이 사라진다.
    업로드일시가 문자열(YYYYMMDDHH24MISS)이라 문자열 정렬이지만, 자릿수가 고정이라
    사전순 = 시간순이 맞다. 형식을 바꾸면 이 성질이 깨진다.
-->
<!-- MySQL: LIMIT ... OFFSET -->
<select id="findByInstitutionNameContaining" databaseId="mysql" resultMap="uploadedFileMap">
    SELECT * FROM INST1.TSGIAF01
    WHERE 기관명 LIKE CONCAT('%', #{keyword}, '%')
    ORDER BY 업로드일시 DESC
    LIMIT #{limit} OFFSET #{offset}
</select>
<!-- Oracle: ROWNUM 2단 중첩 페이징. 안쪽에서 상한, 바깥에서 하한을 건다 -->
<select id="findByInstitutionNameContaining" databaseId="oracle" resultMap="uploadedFileMap">
    SELECT * FROM (
        SELECT a.*, ROWNUM RN FROM (

```

```

        SELECT * FROM INST1.TSKGIAF01
        WHERE 기관명 LIKE '%' || #{keyword} || '%'
        ORDER BY 업로드일시 DESC
    ) a WHERE ROWNUM <= #{offset} + #{limit}
    ) WHERE RN > #{offset}
</select>

<!-- MySQL: CONCAT -->
<select id="countByInstitutionNameContaining" databaseId="mysql" resultType="long">
    SELECT COUNT(*) FROM INST1.TSKGIAF01
    WHERE 기관명 LIKE CONCAT('%', #{keyword}, '%')
</select>

<!-- Oracle: || 로 연결 -->
<select id="countByInstitutionNameContaining" databaseId="oracle" resultType="long">
    SELECT COUNT(*) FROM INST1.TSKGIAF01
    WHERE 기관명 LIKE '%' || #{keyword} || '%'
</select>

<!-- MySQL: 다중 조건 검색 -->
<select id="search" databaseId="mysql" resultMap="uploadedFileMap">
    SELECT * FROM INST1.TSKGIAF01
    WHERE
    (#{institution} = '' OR 기관명 LIKE CONCAT('%', #{institution}, '%'))
    AND (#{year} = '' OR 문서년 = #{year})
    AND (#{keyword} = '' OR 원본파일명 LIKE CONCAT('%', #{keyword}, '%'))
    ORDER BY 업로드일시 DESC
</select>

<!--
Oracle: || 로 연결.
⚠ Oracle 은 빈 문자열을 NULL 로 바꾸므로 `#{x} = ''` 는 **항상 거짓**이다.
"빈 값이면 조건을 걸지 않는다"는 원래 뜻을 살리려면 NULL 검사해야 한다.
(MySQL 쪽은 ''를 값으로 담으므로 `= ''` 가 맞다 - 그래서 문장이 같린다.)
-->
<select id="search" databaseId="oracle" resultMap="uploadedFileMap">
    SELECT * FROM INST1.TSKGIAF01
    WHERE
    (#{institution} IS NULL OR 기관명 LIKE '%' || #{institution} || '%')
    AND (#{year} IS NULL OR 문서년 = #{year})
    AND (#{keyword} IS NULL OR 원본파일명 LIKE '%' || #{keyword} || '%')
    ORDER BY 업로드일시 DESC
</select>

</mapper>

```

src/mapper/InstitutionMapper.xml 86줄

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<!--
⚠ **2026-09-08 - 사내 DB 표준을 반영했다.** 자바 필드명(property)은 그대로다.

    INSTITUTION → TSKGIAF02          시퀀스 → SQ_KGI_AGENTGIGANTABLE

    property   → 컬럼명              인포타입
    id         → 기관일련번호        일련번호16 (DECIMAL)
    name       → 기관명              명40
    category   → 기관영업분류구분    코드2      ★ TypeHandler
    modifiedAt → 시스템사용일시      일시14     ★ TypeHandler (옛 수정일시)
    systemUserNo → 시스템사용자번호  사용자번호7

    ★ TypeHandler 를 resultMap 과 `#{...}` **양쪽에** 적어야 한다. 한쪽만 적으면
    읽거나 쓰기 중 한 방향만 변환돼 조용히 어긋난다. 이유는 UploadedFileMapper 주석.
-->
<mapper namespace="com.kb.uploader.mapper.InstitutionMapper">

    <resultMap id="institutionMap" type="Institution">
        <id property="id" column="기관일련번호"/>
        <result property="name" column="기관명"/>
    
```

```

<result property="category"    column="기관영업분류구분"
        typeHandler="com.kb.uploader.config.InstitutionCategoryTypeHandler"/>
<result property="modifiedAt"  column="시스템사용일시"
        typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
</resultMap>

<sql id="값목록">
    #{name},
    #{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler},
    #{modifiedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}
</sql>

<!--
    ⚠ 같은 id 가 두 번 나오는 자리는 **방언 분기**다(databaseId). 접속한 DB 에 맞는
    쪽만 로드된다 - config/MyBatisConfig 가 값을 정한다. **항상 둘을 같이 고칠 것.**
-->

<!-- MySQL -->
<insert id="insert" databaseId="mysql" useGeneratedKeys="true" keyProperty="id">
    INSERT INTO INST1.TSKGIAF02 (기관명, 기관영업분류구분, 시스템사용일시, 시스템사용자번호)
    VALUES (<include refid="값목록"/>, #{systemUserNo})
</insert>

<!-- Oracle: 시퀀스에서 먼저 ID를 받아 직접 넣는다 (schema-oracle.sql 의 SQ_KGI_AGENTGIGANTABLE) -->
<insert id="insert" databaseId="oracle">
    <selectKey keyProperty="id" resultType="long" order="BEFORE">
        SELECT INST1.SQ_KGI_AGENTGIGANTABLE.NEXTVAL FROM DUAL
    </selectKey>
    INSERT INTO INST1.TSKGIAF02 (기관일련번호, 기관명, 기관영업분류구분, 시스템사용일시, 시스템사용자번호)
    VALUES (#{id}, <include refid="값목록"/>, #{systemUserNo})
</insert>

<update id="update">
    UPDATE INST1.TSKGIAF02
    SET 기관명      = #{name},
        기관영업분류구분 = #{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler},
        시스템사용일시  = #{modifiedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
        시스템사용자번호 = #{systemUserNo}
    WHERE 기관일련번호 = #{id}
</update>

<delete id="deleteById">
    DELETE FROM INST1.TSKGIAF02 WHERE 기관일련번호 = #{id}
</delete>

<delete id="deleteAll">
    DELETE FROM INST1.TSKGIAF02
</delete>

<!--
    ⚠ **이 앱에서 가장 중요한 문장이다.** 업로드된 파일의 기관명으로 여기를 조회해
    못 찾으면 파일이 UNCLASSIFIED 로 남는다(ClassificationService.classify).
    틀리면 오류 없이 "분류가 하나도 안 되는" 증상으로만 나타난다.
-->
<select id="findByName" resultMap="institutionMap">
    SELECT * FROM INST1.TSKGIAF02 WHERE 기관명 = #{name}
</select>

<select id="findAll" resultMap="institutionMap">
    SELECT * FROM INST1.TSKGIAF02 ORDER BY 기관명 ASC
</select>

</mapper>

```

별첨3스키마 SQL 2개 — 전문

①번 6절과 무엇이 다른가

①번은 DBA 가 그대로 실행할 문장만 추린 것이고, 여기는 주석까지 포함한 원본입니다. 주석에 왜 그 길이그 타입인지, 무엇을 일부러 넣지 않았는지가 적혀 있습니다.

두 파일은 같은 커밋에서 함께 고쳐야 합니다

schema-mysql.sql 은 미러입니다. 한쪽만 고치면 외부망 테스트(H2)와 내부망 운영이 조용히 갈립니다. ⚠ Oracle 쪽은 IF NOT EXISTS 가 없어 재실행하면 ORA-00955 — DROP 먼저.

src/schema-oracle.sql 219줄

```
-- =====
-- schema-oracle.sql - uploader 단독 스키마의 Oracle 정보
--
-- 미러 관계: `schema-mysql.sql` 과 **같은 내용**을 Oracle 문법으로 적은 것이다.
--             한쪽이 바뀌면 같은 커밋에서 다른 쪽도 바꾼다
--             (`kgi-ggreport-web/src/main/resources/db/README.md` 와 같은 규칙).
--
-- 왜 필요한가: `config-envs` 의 local-dev.stg 는 이미 `oracle.jdbc.OracleDriver`
--             를 보고 있고 prod 는 JNDI(`jdbc/uploaderDS`)다. MySQL 은 외부망
--             `out-local` 하나뿐인데, 종전에는 DDL 이 `schema-mysql.sql` 한 별뿐이라
--             **내부망 4개 환경에 만들 테이블이 없었다.**
--
-- =====
-- MySQL → Oracle 로 바꾼 것 5가지
--
-- 1. `BIGINT AUTO_INCREMENT` → `NUMBER(18)` + **시퀀스**
--    (자릿수는 2026-09-08 에 19 → 18. 사내 DB 규격에 맞춘 것이고, ID 는 Java `Long`
--    이라 이론상 19자리까지 되지만 시퀀스가 1부터 증가해 실사용에서 무의미하다.)
--    ⚠ IDENTITY(12c+) 가 아니라 시퀀스를 쓴다. 이유 둘:
--    ④ Mapper 의 oracle 분기가 `<selectKey>` 로 `..._SEQ.NEXTVAL` 을 먼저
--    읽어 ID 를 직접 넣는다 - 시퀀스가 실제로 있어야 한다.
--    ⑤ Oracle 11g 에서도 그대로 돈다(버전 회신을 기다릴 필요가 없다).
-- 2. `VARCHAR(n)` → `VARCHAR2(n CHAR)`
--    ⚠ `CHAR` 를 명시한다. 빠뜨리면 NLS 에 따라 **바이트** 의미가 되어
--    한글 기관명이 200자가 아니라 66자에서 잘린다.
-- 3. `DATETIME` → `TIMESTAMP`
--    ⚠ 본체(`kgi-ggreport-web`)의 `007_uploader.sql` 은 같은 열을
--    `VARCHAR2(40 CHAR)` ISO 문자열로 두는데, **일부러 다르게 했다.**
--    007 은 uploader 테이블을 *본체 스키마 안에* 합칠 때의 초안이라 본체의
--    시각 표기 규약(설계 §5)을 따른 것이고, 이 파일은 uploader 가 **단독으로**
--    들 때의 정보이다. uploader 도메인은 `LocalDateTime` 을 쓰고
--    (`UploadedFile.uploadedAt` 등) Mapper 도 문자열 변환을 하지 않으므로,
--    여기서 문자열로 두면 코드를 고쳐야 한다. 단계 6(본체 편입) 때 어느 쪽으로
--    합칠지 정한다 - 그때까지 두 파일은 **의도적으로 공존**한다.
-- 4. `ENGINE=InnoDB` / `DEFAULT CHARSET=utf8mb4` 절 제거 (Oracle 에 없다)
-- 5. `CREATE TABLE IF NOT EXISTS` → `CREATE TABLE`
--    ⚠ Oracle 에 `IF NOT EXISTS` 가 없다. **이 스크립트는 재실행하면
--    ORA-00955(이미 있는 이름)로 실패한다.** 다시 깔려면 파일 끝의
--    DROP 문단을 먼저 돌릴 것.
--
-- ⚠ 이 스키마는 `spring.sql.init` 으로 자동 실행되지 않는다. 그 설정은
-- 테스트(H2)에만 있다 - 내부망에서는 DBA 가 이 파일을 직접 적용한다.
--
-- =====
-- 2026-09-08 - **컬럼은 한글 물리명, 테이블은 코드성 물리명** (사내 표준)
--
-- 내부망 DB 표준이 **컬럼은 한글, 테이블은 코드**(`TSKGIAF01`·`TSKGIAF02`)라 그에 맞췄다.
-- ⚠ 같은 날 영문 서술형(`UPLOADED_FILE_ID` 등)으로 한 번 개명했다가 이 표준을
-- 받고 다시 바꾼 것이다. **그 중간 이름은 커밋 `1e13025`에만 남아 있고
-- 어디에서도 쓰지 않는다** - 옛 문서를 보고 그 이름을 되살리지 말 것.
```

```

--
--      UPLOADED_FILE → TSKGIAF01      INSTITUTION → TSKGIAF02
--
--      ID          → 업로드파일ID      ID          → 기관ID
--      ORIGINAL_NAME → 원본파일명      NAME          → 기관명
--      STORED_PATH  → 저장경로      CATEGORY      → 기관분류
--      FILE_YEAR    → 문서연도      MODIFIED_AT   → 수정일시
--      INSTITUTION_NAME → 기관명
--      CATEGORY      → 기관분류
--      STATUS        → 분류상태
--      UPLOADED_AT   → 업로드일시
--      CLASSIFIED_AT → 분류일시
--
--  ▲ **제약·인덱스·시퀀스는 테이블 코드를 접두로 쓴다**(2026-09-08).
--      테이블이 코드성이 되면서 옛 이름(`PK_UPLOADED_FILE` 등)이 어느 표의 것인지
--      드러나지 않게 됐다. 맵은 이 이름들을 참조하지 않으므로 바뀌도 무해하다
--      - 다만 **시퀀스 이름은 Mapper 가 직접 부르므로 바꾸면 매퍼도 같이 고쳐야
--      한다**(아래 시퀀스 절).
--
--  ▲ **30바이트 상한**: Oracle 11g 의 식별자 상한은 30바이트인데 한글은 AL32UTF8
--      에서 글자당 3바이트라 **한글 10자**가 한계다. 테이블까지 한글로 갔다면
--      `IDX_업로드파일_업로드일시` (35바이트)처럼 상한을 넘는 이름이 생겼겠지만,
--      테이블 코드를 접두로 쓰면 `IX_TSKGIAF01_업로드일시` (29바이트)로 들어간다.
--      컬럼명은 가장 긴 `업로드파일일련번호` (27바이트)도 상한 안에 들어간다.
--
--  ▲ **따옴표를 쓰지 않는다.** Oracle·MySQL·H2 모두 한글 식별자를 무인용으로
--      받는다(MySQL 은 U+0080~U+FFFF 를 무인용 식별자로 명시 허용). 큰따옴표로
--      감싸면 대소문자가 고정돼 도구마다 성가셔진다.
--
--  ▲ **Mapper XML 2개가 이 이름에 묶여 있다**(`UploadedFileMapper.xml`·
--      `InstitutionMapper.xml` 의 resultMap `column` 과 모든 SQL). 한쪽만 고치면
--      기동은 되고 조회에서 `ORA-00904: invalid identifier` 가 난다.
--  =====
--
--  — 업로드된 파일 (TSKGIAF01) —
--
--  ▲ 컬럼 순서는 **조회 편의**로 정했다(2026-09-08). Oracle 에서 물리 컬럼 순서는
--  조회 성능과 사실상 무관하고(9개 컬럼에서는 측정되지 않는다), 앱도 컬럼을
--  **이름으로만** 다룬다(resultMap 이름 매핑 + INSERT 에 컬럼 목록 명시).
--  그래서 사람이 표를 읽는 순서에 맞췄다: **주 조회 축인 기관 → 분류 → 연도 →
--  상태 → 파일명 → 시각**, 그리고 가장 길고 사람이 읽지 않는 저장경로를 맨 뒤로.
--
--  CREATE TABLE INST1.TSKGIAF01 (
--      -- 인포타입 일련번호16 = DECIMAL(16). Oracle 에서 DECIMAL 은 NUMBER 의 동의어다.
--      -- 업로드파일일련번호 DECIMAL(16)      NOT NULL,      -- 일련번호16 · 속성명 동일
--      -- ▲ FK 가 아니라 **이름 문자열**이다. TSKGIAF02.기관명 이 바뀌면 조용히 갱신다.
--      -- 기관명      VARCHAR2(40 CHAR),      -- 명40 · 속성명 동일
--      -- ● **코드값이다.** 01 지방자치단체 / 02 공공기관 / 03 대학교 / 04 병원 / 05 법원
--      --      자바·화면은 한글 이름을 그대로 쓰고 변환은 InstitutionCategoryTypeHandler 가 한다.
--      --      코드표 정본: code/InstitutionCategory.java
--      -- 기관영업분류구분      VARCHAR2(2 CHAR),      -- 구분코드2 · 속성명 기관영업분류구분코드
--      -- 문서년      VARCHAR2(4 CHAR),      -- 년4 · 속성명 동일 · 예 '2026'
--      -- ● **코드값이다.** 01 UNCLASSIFIED / 02 CLASSIFIED / 03 REJECTED / 04 DELETED
--      -- 분류상태구분      VARCHAR2(2 CHAR)  DEFAULT '01' NOT NULL,      -- 구분코드2 · 속성명 분류상태구분코드
--      -- 파일명 상한은 파일시스템이 정한다(NTFS·ext4 모두 255자). 인포타입에 255 가 없어 260.
--      -- 원본파일명      VARCHAR2(260 CHAR) NOT NULL,      -- 명260 · 속성명 동일
--      -- ● **시각이 문자열이다** (사내 표준). 형식 YYYYMMDDHH24MISS 14자로 길이와 정확히 같다.
--      --      자바는 LocalDateTime 을 쓰고 변환은 LocalDateTimeStringTypeHandler 가 한다.
--      -- ▲ 자릿수가 고정이라 **문자열 정렬 = 시간 정렬**이다. 형식을 바꾸면 깨진다.
--      -- 업로드일시      VARCHAR2(14 CHAR)  NOT NULL,      -- 일시14 · 속성명 동일 · **업무 시각**
--      -- 분류일시      VARCHAR2(14 CHAR),      -- 일시14 · 분류 전에는 NULL
--      -- 최약 경로가 약 341자(baseDir 20 + /classified/ 12 + 분류 2 + 년 4 + 기관 40 + 파일명 260 + 구분자 3).
--      -- 저장경로내용      VARCHAR2(600 CHAR) NOT NULL,      -- 내용600 · 속성명 동일
--      -- — 감사 컬럼 2개 (2026-09-08 신설) —
--      -- **업로드일시와 다르다.** 업로드일시는 업무 시각(사용자가 올린 때)이고,
--      -- 이쪽은 **행**을 마지막으로 건드린 시각**이다. 규칙(사용자 확정):
--      --      · INSERT      → 업로드일시 와 같은 값
--      --      · UPDATE(분류) → 분류일시 와 같은 값
--      --      · 삭제·반려    → 그 시점의 DB 시각(TO_CHAR(SYSDATE, 'YYYYMMDDHH24MISS'))
--      -- 시스템사용일시      VARCHAR2(14 CHAR),      -- 일시14 · 속성명 동일
--      -- 이 데이터를 넣게 만든 **주체**다. 사람이 아니라 프로그램이라 화면번호를 넣는다
--      -- (앞 K 를 뺀 7자: KGI11100 → GI11100). 배치는 'BATCH01'.
--      -- 도출은 code/SystemUser.java 가 컨트롤러 클래스명에서 자동으로 한다.

```

```

시스템사용자번호 CHAR(7), -- 사용자번호7 · 속성명 동일
CONSTRAINT PK_TSKGIAF01 PRIMARY KEY (업로드파일일련번호),
CONSTRAINT CK_TSKGIAF01_분류상태 CHECK (분류상태구분 IN ('01', '02', '03', '04'))
);

-- — 기관 (TSKGIAF02) —————
-- ⚠ 이름과 달리 "기관 마스터"가 아니다. 주소·기관코드 같은 기관 정보가 없는 이유가
-- 그것이다. 실제 역할은 둘뿐이다(ClassificationService.classify):
-- ① 기관명 → 기관분류 **매핑표** ② 등록된 기관만 분류되는 **허용목록**
-- 그래서 컬럼 순서도 **입력(기관명) → 출력(기관분류)** 이다.
CREATE TABLE INST1.TSGIAF02 (
    기관일련번호 DECIMAL(16) NOT NULL, -- 일련번호16 · 속성명 동일
    기관명 VARCHAR2(40 CHAR) NOT NULL, -- 명40 · 속성명 동일
    기관영업분류구분 VARCHAR2(2 CHAR) NOT NULL, -- 구분코드2 · 속성명 기관영업분류구분코드
    -- — 아래 4개는 **앱이 읽지도 쓰지도 않는다** — 기관 정보를 미리 담아 둘 자리다.
    -- 도메인 Institution · Mapper · 화면에 대응하는 것이 없고, resultMap 이 모르는
    -- 컬럼은 그냥 무시하며 INSERT 도 컬럼을 명시하므로 NULL 로 남는다.
    협약기관상태구분 VARCHAR2(2 CHAR), -- 구분코드2 · 01 미협약 / 02 협약
    입찰공고상태구분 VARCHAR2(2 CHAR), -- 구분코드2 · 01 공고전 / 02 공고중 / 03 공고완료
    입찰공고시작년월일 VARCHAR2(8 CHAR), -- 년월일8 · YYYYMMDD
    입찰공고마감년월일 VARCHAR2(8 CHAR), -- 년월일8 · YYYYMMDD
    -- ⚠ **옛 `수정일시` 가 이 이름으로 바뀐 것**이다(2026-09-08). 그래서 여기만
    -- NOT NULL 이고(TSGIAF01 쪽은 NULLABLE), 도메인 Institution.modifiedAt 이
    -- 그대로 이 컬럼에 매핑된다.
    시스템사용일시 VARCHAR2(14 CHAR) NOT NULL, -- 일시14 · 속성명 동일
    시스템사용자번호 CHAR(7), -- 사용자번호7 · 속성명 동일
    CONSTRAINT PK_TSGIAF02 PRIMARY KEY (기관일련번호),
    CONSTRAINT UK_TSGIAF02_기관명 UNIQUE (기관명)
);

-- — 시퀀스 —————
-- ⚠ 이름을 바꾸지 말 것. Mapper XML 의 oracle 분기가 이 이름을 그대로 부른다
-- ('`UploadedFileMapper.xml`' / '`InstitutionMapper.xml`' 의 <selectKey>).
-- ⚠ 2026-09-08 사내 표준 검사로 이름이 바뀌었다 — 테이블 코드를 따르던
-- `TSKGIAF01_SEQ` / `TSKGIAF02_SEQ` 에서 `SQ_KGI_...` 로. 22자리 30바이트 상한 안이다.
-- **테이블 이름과 시퀀스 이름이 서로 안 닮았으므로** 어느 표의 것인지는
-- 아래 주석으로만 알 수 있다: AGENTFILE=TSKGIAF01, AGENTGIGAN=TSKGIAF02.
-- 여기만은 이름을 바꾸면
-- **매퍼의 <selectKey> 도 같은 커밋에서 고쳐야 한다** — 제약·인덱스와 다른 점이다.
-- `NOCACHE` 는 WAS 재기동 때 번호가 크게 건너뛰지 않게 하려는 것이다 —
-- ID 에 의미를 두지 않으므로 성능이 문제되면 캐시를 켜도 된다.
CREATE SEQUENCE INST1.SQ_KGI_AGENTFILETABLE START WITH 1 -- TSGIAF01 용 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE INST1.SQ_KGI_AGENTGIGANTABLE START WITH 1 -- TSGIAF02 용 INCREMENT BY 1 NOCACHE;

-- — 인덱스 —————
-- ⚠ 지금 데이터 규모(연 수천 건)에서는 없어도 된다. 수만 행 폴스캔은 밀리초다.
-- 아래 하나만 두는 이유는 비용이 거의 없고 **가장 자주 쓸 접근 경로**를 받기 때문이다.

-- 기관이 주 조회 축이다("A 기관 자료 들어왔나?").
-- ⚠ **열 순서가 이 인덱스의 전부다.** (기관명, 업로드일시) 순서라서
-- 아래 두 쿼리가 **정렬 없이** 처리된다 — 인덱스를 시간순으로 훑으며 읽기만 한다.
--
-- WHERE 기관명 = ? ORDER BY 업로드일시 DESC ← 미래의 기관 검색
-- WHERE 기관명 = ? AND 분류상태구분 = ? ORDER BY 업로드일시 DESC ← findClassifiedByUnknownInstitution
--
-- (기관명, 분류상태구분, 업로드일시) 3열도 검토했으나 **버렸다.** 위 첫 쿼리처럼
-- 분류상태구분을 안 거는 경우, 가운데 열 때문에 인덱스가 시간순이 아니게 되어
-- 정렬이 되살아난다. 두 번째 쿼리의 분류상태구분은 읽으면서 걸러도 싸다.
--
-- ⚠ 화면 검색(findByInstitutionNameContaining)은 `LIKE '%키워드%'` 라
-- **선행 와일드카드 때문에 이 인덱스를 못 쓴다.** 기관 입력을 드롭다운
-- (정확 일치)으로 바꾸는 후속 작업이 끝나야 제 값을 한다.
CREATE INDEX IX_TSGIAF01_기관명일시 ON INST1.TSGIAF01 (기관명, 업로드일시);

-- 상태별 목록 — **앱에서 가장 많이 쓰는 조건**이다. `분류상태구분 = ?` 을 부르는 곳이
-- 여섯이다: 파일처리상태 화면(01) · 분류완료 목록 화면(02) · 재분류 잡(01) ·
-- InstitutionService 두 곳(01).
-- ⚠ 두 번째 열이 PK 라서 **정렬까지 인덱스가 처리**한다
-- (findByStatus 는 ORDER BY 업로드파일일련번호 DESC 다).
-- ⚠ 이름이 30바이트 상한에 걸려 `_분류상태` 로 줄였다(25바이트). `_상태일련번호` 는
-- 31바이트로 **넘는다** — 한글은 글자당 3바이트다.

```

```
CREATE INDEX IX_TSKGIAF01_분류상태 ON INST1.TSKGIAF01 (분류상태구분, 업로드파일일련번호);
```

```
-- 넣지 않은 것 - 조건이 오면 그때 넣는다:
--   · (업로드일시) 단독 - **필요 없어졌다.** 최신순 목록(findRecent)을
--   `ORDER BY 업로드파일일련번호 DESC` 로 바꿔 **PK 인덱스가 정렬을 처리**한다.
--   시퀀스가 NOCACHE 라 증가순 = 삽입순 = 시간순이고, 업로드일시가 문자열(초 단위)
--   이라 **같은 초에 여러 건이면 오히려 일련번호가 정확하다.**
--   ⚠ 시퀀스에 캐시를 켜면 재기동 때 번호가 튀어 이 전체가 깨진다.
--   · (분류상태구분) 단독 - 위 복합 인덱스의 선행 컬럼이라 불필요하다.
--   ⚠ 이 인덱스는 한 번 뺐다가 되돌린 것이다(2026-09-08). "재분류 잠이 깨져 있으니
--   보류"라고 판단했는데, 정작 이 조건을 부르는 것은 잠이 아니라 **화면 두 개**였다.

-- =====
-- 재설치용 DROP (필요할 때만 직접 실행)
--
-- DROP SEQUENCE INST1.SQ_KGI_AGENTFILETABLE;
-- DROP SEQUENCE INST1.SQ_KGI_AGENTGIGANTABLE;
-- DROP TABLE INST1.TSKGIAF01 PURGE;
-- DROP TABLE INST1.TSKGIAF02 PURGE;
--
-- -----
-- 미러와 일부러 다르게 둔 것 (drift 아님)
--
--   · 인덱스: ~~양쪽 다 PK/UK 뿐~~ → **2026-09-08 에 양쪽에 같은 돌을 넣었다**
--   (업로드일시 · 기관명+분류상태). 늘리거나 줄일 때도 **두 파일을 같은
--   커밋에서** 함께 고칠 것. 판단 근거는 위 인덱스 절의 주석에 있다.
-- =====
```

src/schema-mysql.sql 58줄

```
-- =====
-- schema-mysql.sql - uploader 단독 스키마의 MySQL 정본
--
-- 미러 관계: `schema-oracle.sql` 과 **같은 내용**을 MySQL 문법으로 적은 것이다.
--   한쪽이 바뀌면 같은 커밋에서 다른 쪽도 바꾼다.
--   바꾼 이유·인포타임·코드표는 저쪽 주석에 있다 - 여기서는 결과만 맞춘다.
--
-- 이 파일을 쓰는 곳: 외부망 `out-local`(MySQL) 과 테스트 H2(`MODE=MySQL`).
--
-- ⚠ 일부러 다른 것 하나: PK 를 **BIGINT AUTO_INCREMENT** 로 둔다. Oracle 쪽은
--   인포타임 그대로 `DECIMAL(16)` + 시퀀스인데, MySQL 의 AUTO_INCREMENT 는
--   DECIMAL 에 걸 수 없다. 자릿수(16)는 BIGINT 안에 들어가므로 값은 같다.
-- =====

CREATE TABLE IF NOT EXISTS TSKGIAF01 (
  업로드파일일련번호 BIGINT NOT NULL AUTO_INCREMENT,
  기관명 VARCHAR(40),
  -- 코드값: 01 지방자치단체 / 02 공공기관 / 03 대학교 / 04 병원 / 05 법원
  기관영업분류구분 VARCHAR(2),
  문서년 VARCHAR(4),
  -- 코드값: 01 UNCLASSIFIED / 02 CLASSIFIED / 03 REJECTED / 04 DELETED
  분류상태구분 VARCHAR(2) NOT NULL DEFAULT '01',
  원본파일명 VARCHAR(260) NOT NULL,
  -- 시작은 문자열 YYYYMMDDHH24MISS (사내 표준). 자바는 LocalDateTime 그대로.
  업로드일시 VARCHAR(14) NOT NULL,
  분류일시 VARCHAR(14),
  저장경로내용 VARCHAR(600) NOT NULL,
  -- 감사 컬럼 2개. 규칙은 schema-oracle.sql 주석 참조.
  시스템사용일시 VARCHAR(14),
  시스템사용자번호 CHAR(7),
  PRIMARY KEY (업로드파일일련번호),
  -- ⚠ 인덱스를 CREATE TABLE 안에 둔 것은 형식만 Oracle 과 다르다. 분리하면
  --   테스트 H2 가 DB 를 공유해 스크립트를 여러 번 돌릴 때 "Duplicate key name"
  --   으로 깨진다(CREATE TABLE IF NOT EXISTS 는 멍청한데 CREATE INDEX 는 아니다).
  --   열 순서가 전부다 - (기관명, 업로드일시) 라서 기관 조회 + 최신순 정렬이
  --   정렬 없이 처리된다. 근거는 schema-oracle.sql 의 인덱스 절.
  KEY IX_TSKGIAF01_기관명일시 (기관명, 업로드일시),
  -- 상태별 목록 - 앱에서 가장 많이 쓰는 조건(6곳). 두 번째 열이 PK 라 정렬까지 받는다.
  KEY IX_TSKGIAF01_분류상태 (분류상태구분, 업로드파일일련번호),
  -- ⚠ MySQL 은 8.0.16 부터 CHECK 를 실제로 강제한다.
```

```

        CONSTRAINT CK_TSGIAF01_분류상태 CHECK (분류상태구분 IN ('01', '02', '03', '04'))
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS TSGIAF02 (
    기관일련번호      BIGINT      NOT NULL AUTO_INCREMENT,
    기관명            VARCHAR(40) NOT NULL,
    기관영업분류구분  VARCHAR(2)  NOT NULL,
    -- 아래 4개는 애플이 읽지도 쓰지도 않는다(자리만 만들어 둔 것).
    협약기관상태구분  VARCHAR(2),          -- 01 미협약 / 02 협약
    입찰광고상태구분  VARCHAR(2),          -- 01 공고전 / 02 공고중 / 03 공고완료
    입찰광고시작년월일 VARCHAR(8),        -- YYYYMMDD
    입찰광고마감년월일 VARCHAR(8),        -- YYYYMMDD
    -- 옛 수정일시가 이 이름으로 바뀐 것이다. 그래서 여기만 NOT NULL 이다.
    시스템사용일시    VARCHAR(14) NOT NULL,
    시스템사용자번호  CHAR(7),
    PRIMARY KEY (기관일련번호),
    UNIQUE KEY UK_TSGIAF02_기관명 (기관명)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

별첨4수정된 자바·화면 14개 — 전문

별첨1~3 과 무엇이 다른가

별첨1~3 은 새로 만드는 파일(자바 9 · 매퍼 2 · 스키마 2)이고, 여기는 이미 있던 파일을 고친 것입니다. 앞의 5절이 바뀐 줄만 발췌한 것이라, 통째로 옮겨 적으실 때는 이쪽을 기준으로 삼으십시오.

대부분은 한두 줄 변경입니다

컨트롤러 6개와 서비스 3개는 **import 한 줄 + 세팅 한두 줄**이 전부입니다. 실제로 내용이 늘어난 것은 UploadedFile (필드 2개)· UploadedFileMapper (파라미터)· ReclassificationJob (try/finally) 정도입니다.

src/com/kb/uploader/domain/UploadedFile.java 76줄 · systemUserNo · systemUsedAt 필드 추가

```
package com.kb.uploader.domain;

import java.time.LocalDateTime;

public class UploadedFile {

    /** 감사 컬럼 시스템사용자번호. 화면번호(앞 K 제외 7자) 또는 'BATCH01'. code/SystemUser 참조. */
    private String systemUserNo;

    /**
     * 감사 컬럼 시스템사용일시. <b>업로드일시(업무 시각)와 다르다</b> - 이 행을 마지막으로
     * 건드린 시각이다. 규칙(사용자 확정 2026-09-08):
     * 업로드 때는 업로드일시, 분류 때는 분류일시와 <b>같은 값</b>을 쓰고,
     * 그 둘이 없는 경우(삭제·반려·분류일시 없는 수정)는 <b>그 시점의 시각</b>을 쓴다.
     * ⚠ DB 함수로 채우지 않는 이유는 방언이 갈리고(TO_CHAR/DATE_FORMAT) 테스트로
     * 확인하기 어렵기 때문이다 - 자바에서 정해 넘기면 값이 정확히 일치한다.
     */
    private LocalDateTime systemUsedAt;

    private Long id;
    private String originalName;
    private String storedPath;
    private String year;
    private String institutionName;
    private String category;
    private String status;
    private LocalDateTime uploadedAt;
    private LocalDateTime classifiedAt;

    public UploadedFile() {}

    public UploadedFile(String originalName, String storedPath,
                        String year, String institutionName) {
        this.originalName = originalName;
        this.storedPath = storedPath;
        this.year = year;
        this.institutionName = institutionName;
        this.status = "UNCLASSIFIED";
        this.uploadedAt = LocalDateTime.now();
    }

    public void classify(String category, String storedPath) {
        this.category = category;
        this.storedPath = storedPath;
        this.status = "CLASSIFIED";
        this.classifiedAt = LocalDateTime.now();
    }
}
```

```

    public void softDelete() {
        this.status = "DELETED";
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getOriginalName() { return originalName; }
    public void setOriginalName(String originalName) { this.originalName = originalName; }
    public String getStoredPath() { return storedPath; }
    public void setStoredPath(String storedPath) { this.storedPath = storedPath; }
    public String getYear() { return year; }
    public void setYear(String year) { this.year = year; }
    public String getInstitutionName() { return institutionName; }
    public void setInstitutionName(String institutionName) { this.institutionName = institutionName; }
    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
    public LocalDateTime getUploadedAt() { return uploadedAt; }
    public void setUploadedAt(LocalDateTime uploadedAt) { this.uploadedAt = uploadedAt; }
    public LocalDateTime getClassifiedAt() { return classifiedAt; }
    public void setClassifiedAt(LocalDateTime classifiedAt) { this.classifiedAt = classifiedAt; }

    public String getSystemUserNo() { return systemUserNo; }
    public void setSystemUserNo(String systemUserNo) { this.systemUserNo = systemUserNo; }
    public LocalDateTime getSystemUsedAt() { return systemUsedAt; }
    public void setSystemUsedAt(LocalDateTime systemUsedAt) { this.systemUsedAt = systemUsedAt; }
}

```

src/com/kb/uploader/domain/Institution.java 39줄 · systemUserNo 필드 추가

```

package com.kb.uploader.domain;

import java.time.LocalDateTime;

public class Institution {

    /** 감사 컬럼 시스템사용자번호. 화면번호(앞 K 제외 7자) 또는 'BATCH01'. code/SystemUser 참조. */
    private String systemUserNo;

    private Long id;
    private String name;
    private String category;
    private LocalDateTime modifiedAt;

    public Institution() {}

    public Institution(String name, String category) {
        this.name = name;
        this.category = category;
        this.modifiedAt = LocalDateTime.now();
    }

    public void updateCategory(String category) {
        this.category = category;
        this.modifiedAt = LocalDateTime.now();
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }
    public LocalDateTime getModifiedAt() { return modifiedAt; }
    public void setModifiedAt(LocalDateTime modifiedAt) { this.modifiedAt = modifiedAt; }

    public String getSystemUserNo() { return systemUserNo; }

```

```
    public void setSystemUserNo(String systemUserNo) { this.systemUserNo = systemUserNo; }  
}
```

src/com/kb/uploader/mapper/UploadedFileMapper.java 48줄 · softDeleteById-rejectById 에 파라미터 2개

```
package com.kb.uploader.mapper;  
  
import java.time.LocalDateTime;  
import com.kb.uploader.domain.UploadedFile;  
import org.apache.ibatis.annotations.Mapper;  
import org.apache.ibatis.annotations.Param;  
  
import java.util.List;  
import java.util.Optional;  
  
@Mapper  
public interface UploadedFileMapper {  
  
    void insert(UploadedFile file);  
  
    void update(UploadedFile file);  
  
    void softDeleteById(@Param("id") Long id,  
                        @Param("systemUserNo") String systemUserNo,  
                        @Param("systemUsedAt") LocalDateTime systemUsedAt);  
  
    void rejectById(@Param("id") Long id,  
                    @Param("systemUserNo") String systemUserNo,  
                    @Param("systemUsedAt") LocalDateTime systemUsedAt);  
  
    Optional<UploadedFile> findById(@Param("id") Long id);  
  
    List<UploadedFile> findByStatus(@Param("status") String status);  
  
    List<UploadedFile> findClassifiedByUnknownInstitution();  
  
    long countAll();  
  
    long countByStatus(@Param("status") String status);  
  
    List<UploadedFile> findRecent(@Param("limit") int limit);  
  
    List<UploadedFile> findByInstitutionNameContaining(  
        @Param("keyword") String keyword,  
        @Param("offset") int offset,  
        @Param("limit") int limit);  
  
    long countByInstitutionNameContaining(@Param("keyword") String keyword);  
  
    List<UploadedFile> search(@Param("institution") String institution,  
                             @Param("year") String year,  
                             @Param("keyword") String keyword);  
}
```

src/com/kb/uploader/service/FileUploadService.java 86줄 · insert 직전 감사 값 세팅

```
package com.kb.uploader.service;  
  
import com.kb.uploader.code.SystemUser;  
import com.kb.uploader.domain.UploadedFile;  
import com.kb.uploader.dto.ParsedFileName;  
import com.kb.uploader.dto.UploadResultItem;  
import com.kb.uploader.mapper.UploadedFileMapper;  
import org.springframework.stereotype.Service;  
import org.springframework.web.multipart.MultipartFile;  
  
import java.nio.file.Path;  
import java.util.ArrayList;
```

```

import java.util.List;
import java.util.Optional;

@Service
public class FileUploadService {

    private final FileParserService parserService;
    private final FileStorageService storageService;
    private final ClassificationService classificationService;
    private final UploadedFileMapper fileMapper;

    public FileUploadService(FileParserService parserService,
                             FileStorageService storageService,
                             ClassificationService classificationService,
                             UploadedFileMapper fileMapper) {
        this.parserService = parserService;
        this.storageService = storageService;
        this.classificationService = classificationService;
        this.fileMapper = fileMapper;
    }

    public List<UploadResultItem> upload(List<MultipartFile> files) {
        return upload(files, "", "", "");
    }

    public List<UploadResultItem> upload(List<MultipartFile> files,
                                         String instOverride, String yearOverride, String categoryOverride) {
        List<UploadResultItem> results = new ArrayList<>();
        for (MultipartFile file : files) {
            String originalName = file.getOriginalFilename();
            try {
                Optional<ParsedFileName> parsed = parserService.parse(originalName);
                if (!parsed.isPresent()) {
                    boolean hasOverride = instOverride != null && !instOverride.trim().isEmpty();
                    Path saved = storageService.saveToUnclassified(file, originalName);
                    UploadedFile entity = new UploadedFile(
                        originalName, saved.toString(),
                        (yearOverride != null && !yearOverride.trim().isEmpty()) ? yearOverride.trim() : null,
                        hasOverride ? instOverride.trim() : "알수없음");
                    entity.setSystemUserNo(SystemUser.get());
                    entity.setSystemUsedAt(entity.getUploadedAt());
                    fileMapper.insert(entity);
                    if (hasOverride) {
                        boolean ok = classificationService.classify(entity);
                        results.add(new UploadResultItem(originalName, ok,
                            ok ? entity.getCategory() : null,
                            ok ? "수동 입력으로 분류 완료" : "미분류 (기관 미등록 - 기관 관리에서 등록 필요)"));
                    } else {
                        results.add(new UploadResultItem(originalName, false, null,
                            "파일명 형식 불일치 (년도_기관명_설명.확장자 필요)"));
                    }
                }
                continue;
            }
            ParsedFileName p = parsed.get();
            Path saved = storageService.saveToUnclassified(file, originalName);
            UploadedFile entity = new UploadedFile(
                originalName, saved.toString(), p.getYear(), p.getInstitutionName());
            entity.setSystemUserNo(SystemUser.get());
            entity.setSystemUsedAt(entity.getUploadedAt());
            fileMapper.insert(entity);

            boolean ok = classificationService.classify(entity);
            results.add(new UploadResultItem(
                originalName, ok,
                ok ? entity.getCategory() : null,
                ok ? "분류 완료" : "미분류 (기관 미등록)"));
        } catch (Exception e) {
            results.add(new UploadResultItem(originalName, false, null,
                "오류: " + e.getMessage()));
        }
    }
}

```

```

        return results;
    }
}

```

src/com/kb/uploader/service/ClassificationService.java 53줄 · update 직전 감사 값 세팅

```

package com.kb.uploader.service;

import java.time.LocalDateTime;
import com.kb.uploader.code.SystemUser;
import com.kb.uploader.domain.Institution;
import com.kb.uploader.domain.UploadedFile;
import com.kb.uploader.mapper.InstitutionMapper;
import com.kb.uploader.mapper.UploadedFileMapper;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Optional;

@Service
public class ClassificationService {

    private static final Logger log = LoggerFactory.getLogger(ClassificationService.class);

    private final InstitutionMapper institutionMapper;
    private final UploadedFileMapper fileMapper;
    private final FileStorageService storageService;

    public ClassificationService(InstitutionMapper institutionMapper,
                                UploadedFileMapper fileMapper,
                                FileStorageService storageService) {
        this.institutionMapper = institutionMapper;
        this.fileMapper = fileMapper;
        this.storageService = storageService;
    }

    public boolean classify(UploadedFile file) {
        Optional<Institution> inst = institutionMapper.findByName(file.getInstitutionName());
        if (!inst.isPresent()) return false;

        try {
            Path source = Paths.get(file.getStoredPath());
            Path dest = storageService.moveToClassified(
                source, inst.get().getCategory(), file.getYear(), file.getInstitutionName());
            file.classify(inst.get().getCategory(), dest.toString());
            file.setSystemUserNo(SystemUser.get());
            file.setSystemUsedAt(
                file.getClassifiedAt() != null ? file.getClassifiedAt() : LocalDateTime.now());
            fileMapper.update(file);
            return true;
        } catch (Exception e) {
            log.error("파일 분류 실패: {}", file.getOriginalName(), e);
            return false;
        }
    }
}

```

src/com/kb/uploader/service/InstitutionService.java 123줄 · insert-update 3자리에 감사 값

```

package com.kb.uploader.service;

import com.kb.uploader.code.SystemUser;
import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.kb.uploader.domain.Institution;

```

```

import com.kb.uploader.domain.UploadedFile;
import com.kb.uploader.mapper.InstitutionMapper;
import com.kb.uploader.mapper.UploadedFileMapper;
import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.io.*;
import java.util.*;
import java.util.stream.Collectors;

@Service
public class InstitutionService {

    private static final ObjectMapper MAPPER = new ObjectMapper();

    private final InstitutionMapper institutionMapper;
    private final ClassificationService classificationService;
    private final UploadedFileMapper uploadedFileMapper;

    public InstitutionService(InstitutionMapper institutionMapper,
                             ClassificationService classificationService,
                             UploadedFileMapper uploadedFileMapper) {
        this.institutionMapper = institutionMapper;
        this.classificationService = classificationService;
        this.uploadedFileMapper = uploadedFileMapper;
    }

    public List<Institution> findAll() {
        return institutionMapper.findAll();
    }

    public Institution save(String name, String category) {
        Optional<Institution> existing = institutionMapper.findByName(name);
        if (existing.isPresent()) {
            existing.get().updateCategory(category);
            existing.get().setSystemUserNo(SystemUser.get());
            institutionMapper.update(existing.get());
            return existing.get();
        }
        Institution inst = new Institution(name, category);
        inst.setSystemUserNo(SystemUser.get());
        institutionMapper.insert(inst);
        return inst;
    }

    public void deleteById(Long id) {
        institutionMapper.deleteById(id);
    }

    public byte[] exportToJson() throws IOException {
        Map<String, List<String>> grouped = institutionMapper.findAll().stream()
            .collect(Collectors.groupingBy(
                Institution::getCategory,
                Collectors.mapping(Institution::getName, Collectors.toList())));
        return MAPPER.writerWithDefaultPrettyPrinter().writeValueAsBytes(grouped);
    }

    @Transactional
    public void importFromJson(byte[] json) throws IOException {
        Map<String, List<String>> data = MAPPER.readValue(json,
            new TypeReference<Map<String, List<String>>>() {});
        institutionMapper.deleteAll();
        data.forEach((category, names) -> {
            names.forEach(name -> {
                Institution created = new Institution(name, category);
                created.setSystemUserNo(SystemUser.get());
                institutionMapper.insert(created);
            });
        });
        List<UploadedFile> pending = uploadedFileMapper.findByStatus("UNCLASSIFIED");
    }

```

```

        pending.forEach(f -> classificationService.classify(f));
    }

    public byte[] exportToXlsx() throws IOException {
        try (Workbook wb = new XSSFWorkbook()) {
            Sheet sheet = wb.createSheet("기관목록");
            Row header = sheet.createRow(0);
            header.createCell(0).setCellValue("기관명");
            header.createCell(1).setCellValue("카테고리");
            int rowNum = 1;
            for (Institution inst : institutionMapper.findAll()) {
                Row row = sheet.createRow(rowNum++);
                row.createCell(0).setCellValue(inst.getName());
                row.createCell(1).setCellValue(inst.getCategory());
            }
            ByteArrayOutputStream out = new ByteArrayOutputStream();
            wb.write(out);
            return out.toByteArray();
        }
    }

    @Transactional
    public void importFromXlsx(byte[] xlsx) throws IOException {
        try (Workbook wb = new XSSFWorkbook(new ByteArrayInputStream(xlsx))) {
            Sheet sheet = wb.getSheetAt(0);
            institutionMapper.deleteAll();
            for (int i = 1; i <= sheet.getLastRowNum(); i++) {
                Row row = sheet.getRow(i);
                if (row == null) continue;
                Cell nameCell = row.getCell(0);
                Cell categoryCell = row.getCell(1);
                if (nameCell == null) continue;
                String name = nameCell.getStringCellValue().trim();
                String category = categoryCell != null
                    ? categoryCell.getStringCellValue().trim() : "";
                if (!name.isEmpty()) {
                    Institution created = new Institution(name, category);
                    created.setSystemUserNo(SystemUser.get());
                    institutionMapper.insert(created);
                }
            }
        }
        List<UploadedFile> pending = uploadedFileMapper.findByStatus("UNCLASSIFIED");
        pending.forEach(f -> classificationService.classify(f));
    }
}

```

src/com/kb/uploader/job/ReclassificationJob.java 65줄 · BATCH01 세팅 + finally 해제

```

package com.kb.uploader.job;

import com.kb.uploader.code.SystemUser;
import com.kb.uploader.domain.UploadedFile;
import com.kb.uploader.mapper.UploadedFileMapper;
import com.kb.uploader.service.ClassificationService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

import java.util.List;

/**
 * 미분류로 남은 파일을 다시 분류해 본다. <b>언제 도는지는 여기서 정하지 않는다.</b>
 *
 * <p>⚠ 종전에는 이 메서드에 {@code @Scheduled(cron = "${reclassification.cron}")} 이 붙어 있었다(2026-08-31 제거). 그러면 Spring 이 자기 타이머 스레드 풀을 만들고, <b>WAS 에서 애플이 스레드를 만드는 것은 금지</b>다(설계 §2.§4). 반복 실행은 {@link ReclassificationTrigger} 가 {@link BackgroundScheduler} 를 통해 건다 - WebLogic 에서는 CommonJ TimerManager 가 그 스레드를 준다.
 *
 */

```

```

* <p>덕분에 이 클래스는 <b>"한 번 돌리는 일"만</b> 안다. 수동 실행이 필요해지면
* 그냥 부르면 된다.
*/
@Component
public class ReclassificationJob {

    private static final Logger log = LoggerFactory.getLogger(ReclassificationJob.class);

    private final UploadedFileMapper fileMapper;
    private final ClassificationService classificationService;

    public ReclassificationJob(UploadedFileMapper fileMapper,
                              ClassificationService classificationService) {
        this.fileMapper = fileMapper;
        this.classificationService = classificationService;
    }

    public void reclassify() {
        // ⚠ 배치는 컨트롤러가 없어 SystemUserInterceptor 가 안 돈다. 직접 넣고
        // finally 에서 반드시 지운다 - ThreadLocal 이라 안 지우면 그 스레드를
        // 물려받은 다음 작업에 'BATCH01' 이 새어 나간다.
        SystemUser.set(SystemUser.BATCH);
        try {
            List<UploadedFile> unclassified = fileMapper.findByStatus("UNCLASSIFIED");
            if (unclassified.isEmpty()) {
                log.debug("재분류할 파일 없음");
                return;
            }
            int success = 0;
            for (UploadedFile file : unclassified) {
                try {
                    if (classificationService.classify(file)) {
                        success++;
                    }
                } catch (Exception e) {
                    log.warn("재처리 실패: {} - {}", file.getOriginalName(), e.getMessage());
                }
            }
            log.info("재분류 완료: {}건 성공 / {}건 대상", success, unclassified.size());
        } finally {
            SystemUser.clear();
        }
    }
}

```

src/com/kb/uploader/controller/KGI12000\$FileStatusList.java 34줄 · 분류 목록 하드코딩 제거

```

package com.kb.uploader.controller;

import com.kb.uploader.code.InstitutionCategory;
import com.kb.uploader.domain.UploadedFile;
import com.kb.uploader.mapper.UploadedFileMapper;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;

import java.util.List;

@Controller
@RequestMapping("/file-status")
public class KGI12000$FileStatusList {

    private final UploadedFileMapper fileMapper;

    public KGI12000$FileStatusList(UploadedFileMapper fileMapper) {
        this.fileMapper = fileMapper;
    }
}

```

```

    @GetMapping
    public String execute(Model model) {
        List<UploadedFile> unclassified = fileMapper.findByStatus("UNCLASSIFIED");
        List<UploadedFile> unknownFiles = fileMapper.findClassifiedByUnknownInstitution();
        model.addAttribute("files", unclassified);
        model.addAttribute("unknownFiles", unknownFiles);
        model.addAttribute("categories", InstitutionCategory.names());
        model.addAttribute("unclassifiedCount", (long) unclassified.size());
        return "file-status";
    }
}

```

src/com/kb/uploader/controller/KGI12100\$FileStatusClassify.java 65줄 · 감사 값 세팅

```

package com.kb.uploader.controller;

import java.time.LocalDateTime;
import com.kb.uploader.code.SystemUser;
import com.kb.uploader.mapper.UploadedFileMapper;
import com.kb.uploader.service.FileStorageService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/file-status")
public class KGI12100$FileStatusClassify {

    private static final Logger log = LoggerFactory.getLogger(KGI12100$FileStatusClassify.class);

    private final UploadedFileMapper fileMapper;
    private final FileStorageService storageService;

    public KGI12100$FileStatusClassify(UploadedFileMapper fileMapper,
                                       FileStorageService storageService) {
        this.fileMapper = fileMapper;
        this.storageService = storageService;
    }

    @PostMapping("/{id}/classify")
    public String execute(@PathVariable Long id,
                         @RequestParam String category,
                         @RequestParam(defaultValue = "") String institution,
                         @RequestParam(defaultValue = "") String year,
                         RedirectAttributes ra) {
        fileMapper.findById(id).ifPresent(file -> {
            if (!institution.trim().isEmpty()) {
                file.setInstitutionName(institution.trim());
            }
            if (!year.trim().isEmpty()) {
                file.setYear(year.trim());
            }
            String instName = file.getInstitutionName() != null ? file.getInstitutionName() : "알수없음";
            try {
                java.nio.file.Path source = java.nio.file.Paths.get(file.getStoredPath());
                java.nio.file.Path dest = storageService.moveToClassified(
                    source,
                    category,
                    file.getYear() != null ? file.getYear() : "미확인",
                    instName
                );
                file.classify(category, dest.toString());
                file.setSystemUserNo(SystemUser.get());
                file.setSystemUsedAt(
                    file.getClassifiedAt() != null ? file.getClassifiedAt() : LocalDateTime.now());
            }

```

```

        fileMapper.update(file);
    } catch (Exception e) {
        log.warn("수동 분류 처리 실패: {}", file.getOriginalName(), e);
    }
});
ra.addFlashAttribute("message", "분류 처리 완료");
return "redirect:/file-status";
}
}

```

src/com/kb/uploader/controller/KGI12200\$FileStatusDelete.java 28줄 · 매퍼 호출 인자 2개 추가

```

package com.kb.uploader.controller;

import java.time.LocalDateTime;
import com.kb.uploader.code.SystemUser;
import com.kb.uploader.mapper.UploadedFileMapper;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/file-status")
public class KGI12200$FileStatusDelete {

    private final UploadedFileMapper fileMapper;

    public KGI12200$FileStatusDelete(UploadedFileMapper fileMapper) {
        this.fileMapper = fileMapper;
    }

    @PostMapping("/{id}/delete")
    public String execute(@PathVariable Long id, RedirectAttributes ra) {
        fileMapper.softDeleteById(id, SystemUser.get(), LocalDateTime.now());
        ra.addFlashAttribute("message", "파일이 목록에서 제거되었습니다.");
        return "redirect:/file-status";
    }
}

```

src/com/kb/uploader/controller/KGI12300\$FileStatusReject.java 28줄 · 같음

```

package com.kb.uploader.controller;

import java.time.LocalDateTime;
import com.kb.uploader.code.SystemUser;
import com.kb.uploader.mapper.UploadedFileMapper;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/file-status")
public class KGI12300$FileStatusReject {

    private final UploadedFileMapper fileMapper;

    public KGI12300$FileStatusReject(UploadedFileMapper fileMapper) {
        this.fileMapper = fileMapper;
    }

    @PostMapping("/{id}/reject")
    public String execute(@PathVariable Long id, RedirectAttributes ra) {
        fileMapper.rejectById(id, SystemUser.get(), LocalDateTime.now());
        ra.addFlashAttribute("message", "반려 처리되었습니다.");
    }
}

```

```

        return "redirect:/file-status";
    }
}

```

src/com/kb/uploader/controller/KGI12400\$ClassifiedFileList.java 33줄 · 분류 목록 하드코딩 제거

```

package com.kb.uploader.controller;

import com.kb.uploader.code.InstitutionCategory;
import com.kb.uploader.domain.UploadedFile;
import com.kb.uploader.mapper.UploadedFileMapper;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;

import java.util.List;

@Controller
@RequestMapping("/file-status/classified")
public class KGI12400$ClassifiedFileList {

    private final UploadedFileMapper fileMapper;

    public KGI12400$ClassifiedFileList(UploadedFileMapper fileMapper) {
        this.fileMapper = fileMapper;
    }

    @GetMapping
    public String execute(Model model) {
        List<UploadedFile> classifiedFiles = fileMapper.findByStatus("CLASSIFIED");
        long unclassified = fileMapper.countByStatus("UNCLASSIFIED");
        model.addAttribute("classifiedFiles", classifiedFiles);
        model.addAttribute("categories", InstitutionCategory.names());
        model.addAttribute("unclassifiedCount", unclassified);
        return "classified-rework";
    }
}

```

src/com/kb/uploader/controller/KGI13000\$InstitutionList.java 31줄 · 화면에 분류 목록 전달

```

package com.kb.uploader.controller;

import com.kb.uploader.code.InstitutionCategory;
import com.kb.uploader.mapper.UploadedFileMapper;
import com.kb.uploader.service.InstitutionService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
@RequestMapping("/institutions")
public class KGI13000$InstitutionList {

    private final InstitutionService institutionService;
    private final UploadedFileMapper fileMapper;

    public KGI13000$InstitutionList(InstitutionService institutionService,
                                     UploadedFileMapper fileMapper) {
        this.institutionService = institutionService;
        this.fileMapper = fileMapper;
    }

    @GetMapping
    public String execute(Model model) {
        model.addAttribute("institutions", institutionService.findAll());
    }
}

```

```

        model.addAttribute("categories", InstitutionCategory.names());
        model.addAttribute("unclassifiedCount", fileMapper.countByStatus("UNCLASSIFIED"));
        return "institution";
    }
}

```

src/templates/institution.html 83줄 · 옛 분류 4종 option 제거

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      th:replace="layout :: layout(~{::title}, '기관 카테고리 관리', ~{::section})">
<head><title>기관 카테고리 관리</title></head>
<body>
<section>

    <!-- 페이지 타이틀 -->
    <div class="section-header" style="margin-bottom:20px">
        <span class="section-title" style="font-size:16px">기관 카테고리 관리</span>
    </div>

    <!-- 성공 메시지 -->
    <div th:if="${message}" class="alert-success" th:text="${message}"></div>

    <!-- 내보내기 / 가져오기 툴바 -->
    <div class="toolbar">
        <a th:href="@{/institutions/export/json}" class="btn btn-outline btn-sm">JSON 다운로드</a>
        <a th:href="@{/institutions/export/xlsx}" class="btn btn-outline btn-sm">XLSX 다운로드</a>

        <form th:action="@{/institutions/import/json}" method="post"
              enctype="multipart/form-data" class="form-inline">
            <input type="file" name="file" accept=".json" class="form-control form-control-sm" style="width:180px">
            <button type="submit" class="btn btn-outline btn-sm">JSON 업로드</button>
        </form>

        <form th:action="@{/institutions/import/xlsx}" method="post"
              enctype="multipart/form-data" class="form-inline">
            <input type="file" name="file" accept=".xlsx" class="form-control form-control-sm" style="width:180px">
            <button type="submit" class="btn btn-outline btn-sm">XLSX 업로드</button>
        </form>
    </div>

    <!-- 기관 추가 카드 -->
    <div class="card" style="max-width:520px;margin-bottom:24px">
        <div class="card-header">기관 추가</div>
        <div class="card-body">
            <form th:action="@{/institutions}" method="post" class="form-inline">
                <input type="text" name="name" class="form-control form-control-sm"
                      style="flex:1;min-width:160px" placeholder="기관명" required>
                <select name="category" class="form-control form-control-sm" style="width:150px" required>
                    <option value="">카테고리 선택</option>
                    <option th:each="cat : ${categories}" th:value="${cat}" th:text="${cat}"></option>
                </select>
                <button type="submit" class="btn btn-primary btn-sm">추가</button>
            </form>
        </div>
    </div>

    <!-- 기관 목록 -->
    <div class="section-header">
        <div>
            <span class="section-title">기관 목록</span>
            <span class="section-sub" th:text="'총 ' + ${#lists.size(institutions)} + '개'"></span>
        </div>
    </div>
    <div class="table-card">
        <table>
            <thead>
                <tr>
                    <th>기관명</th><th>카테고리</th><th>수정일</th><th></th>
                </tr>
            </thead>
        </table>
    </div>

```

```

</thead>
<tbody>
  <tr th:each="inst : ${institutions}">
    <td th:text="${inst.name}" style="font-weight:500"></td>
    <td><span class="category-chip" th:text="${inst.category}"></span></td>
    <td class="time-col" th:text="${#temporals.format(inst.modifiedAt, 'yyyy-MM-dd HH:mm')}"></td>
    <td style="width:80px">
      <form th:action="@{/institutions/{id}/delete(id=${inst.id})}"
        method="post">
        <button type="submit" class="btn btn-danger-outline btn-sm"
          onclick="return confirm('삭제하시겠습니까?')">삭제</button>
      </form>
    </td>
  </tr>
</tbody>
</table>
</div>

</section>
</body>
</html>

```

② uploader 소스 변경 상세 · 2026-09-08 확정본 · 선행 문서: [내부망작업지시_2026-09-08_1_스키마변경.pdf](#)